

# Reading Between the Dots: Decoding Hidden Computation across Filler Tokens

**Kaley Brauer\***  
Harvard University  
Cambridge Boston Alignment Initiative

**Claudio Mayrink Verdun**  
Massachusetts Institute of Technology  
Harvard University

**Samuel Marks**  
Anthropic

## Abstract

Frontier LLMs can perform multi-step reasoning over content-free filler tokens like dots or counting sequences, producing correct answers with no visible chain-of-thought (CoT). This is a limit case for behavioral oversight, where surface tokens carry no information about the underlying reasoning. But hidden from the output is not the same as hidden from us. On four task families (fact retrieval, parallel numeric composition, string manipulation, and in-context computation), two open-weights frontier models (DeepSeek V3, Kimi K2) compute over filler tokens in a legible way: attention routes the question through the filler region to the answer, logit-lens readouts show retrieved facts emerging early and their composition crystallizing in late layers, and KV-cache transplants at filler positions causally swap outputs between examples. We introduce an unsupervised decoding pipeline that takes only hidden states as input and recovers intermediate values with 82–94% accuracy (best LLM judge) across both models and all four tasks, without ground-truth labels or training. Even without a judge, the hidden values are already directly in the pipeline’s top-2 tokens 35–85% of the time. The uplift persists whether the filler is prefilled or the model generates the filler itself. On these cleanly decomposable tasks, hidden computation that defeats behavioral CoT monitoring is readable from the residual stream, which suggests that monitorability is a property of the model’s full computational trace rather than only its surface tokens.

## 1 Introduction

Chain-of-thought (CoT) monitorability, the idea that we can audit what a model is doing by reading what it writes, is only as reliable as the assumption that those outputs reflect the underlying computation. Recent calls to preserve this property [1] flag the worry that models may reason in ways CoT-reading cannot catch, whether through optimization pressure toward encoded or obfuscated CoTs or other mechanisms that decouple visible tokens from computation. Filler-token computation [2, 3], where LLMs perform meaningful reasoning over content-free tokens like dots (. . . .) or counting sequences (1 2 3 4 5), is a concrete example of this failure because the tokens do not reflect the computation being done.

Behavioral monitoring is not the only auditing tool available, however. A token that is semantically empty to a reader can still carry decodable information in the residual stream, so what is hidden from the output need not be hidden from an auditor with access to the model’s hidden states.

\*kaley.brauer@cfa.harvard.edu

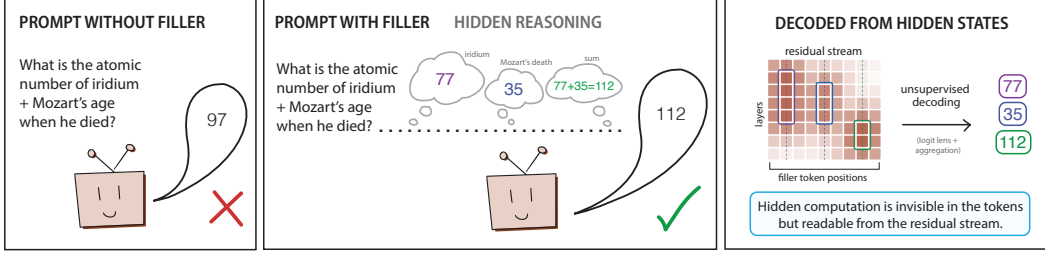


Figure 1: **Frontier-scale LLMs perform decodable hidden multi-step reasoning over meaningless filler tokens.** Without filler the model fails (left); with filler, it succeeds (middle); our unsupervised pipeline decodes the intermediates ( $A_1 = 77$ ,  $A_2 = 35$ ) and their sum (112) from the residual stream at filler positions (right).

We demonstrate this with hidden computation over filler tokens. We study large open-weights models (DeepSeek V3, 671B-parameter with  $\sim 37$ B activated per token; Kimi K2, 1T-parameter with  $\sim 32$ B activated per token) on four task families with a clear intermediate value: 1-fact addition (“What is the age at which Mozart died plus 93?”), 2-fact addition (“What is the atomic number of tungsten plus the atomic number of carbon?”), 2-hop letter-position (“What is the last letter of the name of the capital of the Indian state of Haryana?”), and a system of equations (“If  $ab=51$ ;  $da=2ab-21$ ; ... what is  $3da-79$ ?”), where every intermediate is computed in context rather than retrieved. On each task, we append content-free filler tokens (dots, counting sequences, alphabet sequences) between the question and the answer slot, and ask: does the model do better with filler, and if so, can we recover what it computed at the filler positions?

**Contributions.** The paper moves from establishing the phenomenon, to localizing its mechanism, to recovering its content (Figure 1).

(1) *Hidden computation over filler tokens is a general phenomenon.* Prior work documented filler uplift in commercial frontier models on multi-hop problems [4, 5]. We show the phenomenon is broader than previously documented: open-source models DeepSeek V3 and Kimi K2 benefit from filler tokens on 1-fact addition (54%  $\rightarrow$  72% on DeepSeek V3), 2-fact addition (21%  $\rightarrow$  24% on DeepSeek V3, with 9% of examples flipping wrong  $\rightarrow$  right; 27%  $\rightarrow$  35% on Kimi K2), 2-hop letter position (61%  $\rightarrow$  70% on DeepSeek V3; 69%  $\rightarrow$  75% on Kimi K2), and a system of equations (31%  $\rightarrow$  61% on DeepSeek V3; 18%  $\rightarrow$  36% on Kimi K2). Dots, counting, and alphabet filler all give similar uplift while scrambled sequences underperform, so what matters is a non-disruptive token sequence, not specific content. The uplift extends to the reasoning-trained Kimi K2.5 (2-fact: 39%  $\rightarrow$  48%; system of equations: 31%  $\rightarrow$  46%) and persists when the model generates the filler itself.

(2) *Filler positions causally carry task-relevant information and encode the intermediate computation steps.* Attention analysis shows that with filler present, the model forms a processing relay of question  $\rightarrow$  filler  $\rightarrow$  answer that replaces the answer’s direct line to the question. Logit-lens readouts reveal what the filler region encodes: across tasks and models, the intermediate values appear in the residual stream; in 2-fact addition,  $A_1$  is encoded early in the filler,  $A_2$  later, and the sum  $A_1 + A_2$  crystallizes in late layers immediately before the answer. KV-cache transplants *at only the filler-token rows* establish that this content is causal: transplanting the filler region between matched examples drives the donor’s answer up by tens to hundreds of ranks, for retrieved facts and for values computed in context, and transplanting only the positions where an addend is decoded recovers most of the effect while the complementary positions carry almost none.

(3) *An unsupervised pipeline recovers intermediate values from filler hidden states.* We apply the logit lens at every (layer, filler position), subtract the cross-example mean to remove artifacts, aggregate residual scores across positions, and hand the top-50 tokens to an LLM judge for interpretation. The pipeline requires no ground-truth labels and no training. Without a judge, the target intermediate (a retrieved fact, a pair of parallel numeric intermediates, a string-valued intermediate such as an element or city name, or an in-context-computed value) appears among the top-2 decoded tokens in 35–85% of examples across both models and all four task families; with an LLM judge interpreting the top-50 list, recovery rises to 82–94%. Controls in which the judge sees only the numeric tokens,

or the list with the true intermediates removed, show that the judge’s gain comes from selecting among candidates genuinely present in the list rather than from task priors.

Code and aggregated decoding outputs are available at <https://github.com/kaleybrauer/filler-token-reasoning>.

## 2 Related work

**CoT faithfulness and monitorability.** A growing literature establishes that chain-of-thought traces can fail to reflect the factors driving model behavior. Turpin et al. [6] showed that models produce plausible CoTs that omit biasing features inserted into few-shot examples; Arcuschin et al. [7] extended this to realistic prompts without artificial bias, finding implicit post-hoc rationalization on comparison questions; and Chen et al. [8] found that even modern reasoning models often fail to verbalize prompt-injected hints they relied on, with reveal rates often below 20% in tested settings but substantially higher in others. Baker et al. [9] showed a related dynamic from the training side: strong optimization against a CoT monitor can produce obfuscated reward hacking where the model hides its intent inside CoTs that the monitor accepts. On the other side, Emmons et al. [10] argue that for tasks where CoT is necessary, models struggle to evade monitors without significant help, and propose monitorability rather than faithfulness as the key property. Korbak et al. [1] combine these threads into a call to preserve CoT monitorability as a fragile safety property. Filler-token computation is a concrete failure mode for this property because the surface form of the reasoning is completely unrelated to the computation.

**Filler tokens and hidden computation.** Lanham et al. [2] showed that replacing CoT with filler periods in commercial LLMs gave no benefit, suggesting off-the-shelf models do not use filler tokens. Pfau et al. [3] clarified this was not an in-principle limitation: small transformers *can* learn to use filler tokens, but only with task-specific parallelizable CoT supervision. Goyal et al. [11] found that pretraining with explicit pause tokens yields gains across benchmarks, while introducing them only at finetuning gives mixed results. Greenblatt [4, 5] overturned this view at frontier scale: recent LLMs (Gemini 2.5/3 Pro, Claude Opus 4/4.5) emergently benefit from filler tokens on multi-hop tasks without filler-specific training, with 300 counting tokens nearly doubling Gemini 3 Pro’s 3-hop accuracy (18% → 34%). Filler positions are functionally distinct from attention sinks [12], which absorb excess softmax mass rather than carry information; our KV-cache transplants (Section 4) show filler content is causally example-specific.

**Latent multi-hop reasoning.** A separate line of work studies whether transformers compose facts internally without writing intermediates. Yang et al. [13] find that LLMs internally identify the bridge entity on prompts like “the mother of the singer of ‘Superstition’ is. . .,” with weaker evidence for the second hop; Yang et al. [14] extend this under shortcut-controlled evaluation; and Biran et al. [15] localize the bridge entity to early layers and the second hop to later layers, with a late-to-early patch recovering up to 66% of previously incorrect answers. These works study latent composition without filler tokens; we study what additional structure appears when filler tokens are inserted as a substrate for additional computation. A related line of work examines explicit and internalized CoT tokens. Zhu et al. [16] fine-tune a 1.5B model on synthetic tasks and show that explicit CoT intermediates are causally used and can be stored in trained latent tokens; Huang et al. [17] theoretically analyze a training procedure that internalizes CoT across padding positions and depth; Katz et al. [18] ask models to articulate implicit reasoning steps without inspecting internal representations; and Afzal et al. [19] train a supervised probe to predict whether a zero-shot CoT will succeed. None of these studies the emergent, untrained use of task-irrelevant filler positions in pretrained frontier-scale models, or decodes what those positions carry without labels or training.

**Logit lens and residual-stream interpretability.** The logit lens [20] projects intermediate residual states through the model’s unembedding to obtain a token distribution at each (layer, position). Variants and refinements include tuned-lens [21], future lens [22], and the patchscope framework [23] which generalizes this idea by patching hidden states into a separate prompt that elicits an interpretation. Our pipeline uses the logit lens to surface candidate tokens, cross-example residual subtraction to remove formatting noise, and an LLM judge to interpret, which makes it robust to mid-layer noise and able to name compositional intermediates (e.g., element or city names).

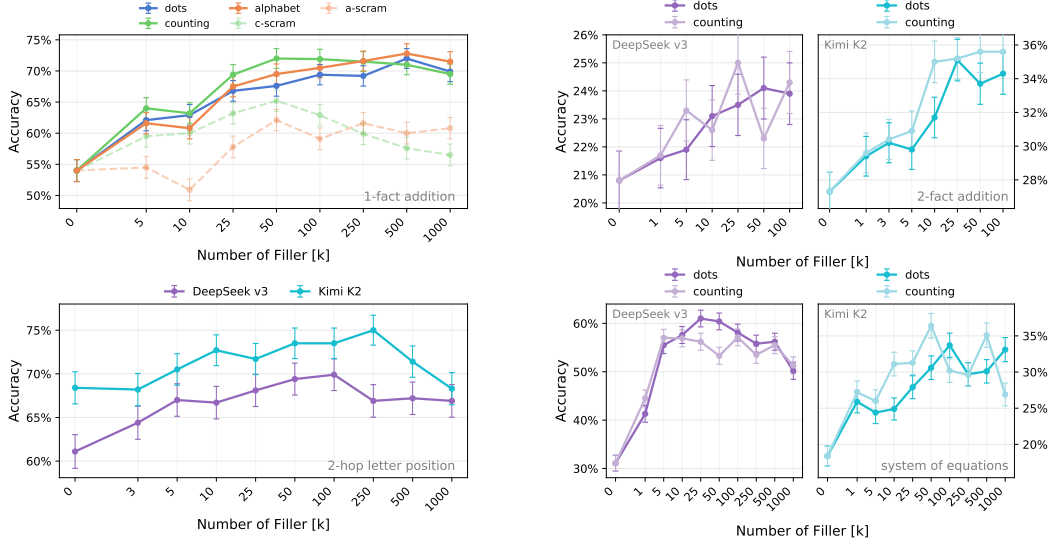


Figure 2: **Filler tokens improve accuracy across tasks, models, and filler types.** Uplift increases with filler length until, at very long lengths, it asymptotes and degrades. Error bars:  $\pm 1$  SE (binomial; same fixed test set across  $k$ ). *Top left*: 1-fact addition on DeepSeek V3 (API; 800 examples); all filler types give comparable uplift while scrambled variants underperform. *Top right*: 2-fact addition on DeepSeek V3 and Kimi K2 (4-bit quantized; 1500 examples). *Bottom left*: 2-hop letter position on both models (API; 637 examples; dot filler). *Bottom right*: system of equations on both models (API; 800 examples).

### 3 Behavioral exploration of filler-token uplift

We first establish that filler tokens improve open-weights model performance across our four task families (examples in Section 1; full task details in Appendix A). Each task has well-defined intermediate value(s). We use  $A$  for a single retrieved fact (1-fact addition),  $A_1, A_2$  for the two retrieved values in 2-fact addition, and  $X$  for a random two-digit addend supplied in the prompt. The intermediate we aim to recover is  $A$  (1-fact), the pair  $\{A_1, A_2\}$  and their sum (2-fact), a string-valued entity such as a city or element name (2-hop letter position), or, for the system of equations, the middle variable  $y$  in the chain  $x \rightarrow c_1x \rightarrow y \rightarrow c_2y \rightarrow \text{answer}$ .

We evaluate DeepSeek V3 and Kimi K2. For each task, the prompt has the form [question] [filler] Answer:. We vary the filler type (dots: “. . . .”; counting: “1 2 3 4 5”; alphabet: “a b c d e”; and scrambled variants c-scam, a-scam) and vary the filler length  $k$ . We use 5 few-shot examples that themselves contain filler. For full task details and example prompts, see Appendix A. The number of filler tokens  $k$  counts appended dots/numbers/letters, not model tokens; tokenization differs by filler type (e.g., “5 ” is two tokens, “.” is one).

Figure 2 shows the results. On 1-fact addition, DeepSeek climbs from a 54% baseline to  $\sim 72\%$  with 100–500 dots/counting/alphabet tokens; scrambled fillers plateau  $\sim 10\%$  lower. On 2-fact addition, DeepSeek climbs from 21.1% to 24.1% with 50 dot tokens, and Kimi climbs from 27.3% to 35.6% with 50 counting tokens. On 2-hop letter position, dot and counting filler both lift DeepSeek from 61% to 70%, with comparable gains for Kimi. On the system of equations, DeepSeek V3 climbs from 31.1% to 61.0% and Kimi K2 from 18.4% to 36.4%, with every filler length giving a statistically significant uplift over baseline. Although DeepSeek’s net 2-fact uplift is small, filler flips  $\sim 9\%$  of examples from wrong to right (counting filler: 135 of 1500; McNemar  $p \approx 5 \times 10^{-4}$  [24]). The uplift is robust to API versus local quantized evaluation (Appendix B). Qwen 3 480B shows a small consistent increase on letter-position (43%  $\rightarrow$  47%) but not on addition, so the phenomenon is broader than previously documented but not universal among comparably-scaled MoE models. Filler tokens placed before the question do not give uplift on these tasks; observed uplift requires filler between the question and answer.

**Reasoning-trained models and model-generated filler.** DeepSeek V3 and Kimi K2 are not RL-trained for chain-of-thought. Filler uplift has been reported for reasoning models such as Gemini 3 Pro and Claude Opus 4 [4]; we add an open-weights case, Kimi K2.5 [25], with explicit reasoning suppressed by prefilling an empty `<think></think>` block. It shows uplift on 2-fact addition (39%  $\rightarrow$  48%) and on the system of equations (31%  $\rightarrow$  46%;  $n = 1,000$  per task and condition). The filler also need not be prefilled. Given few-shot examples and an instruction not to reason in words, Kimi K2.5 generates the filler itself in 71% of attempts, and on those, system-of-equations accuracy rises from 28.7% to 39.2% (25 self-generated dots) and 45.8% (25 counting tokens; Appendix O).

The behavioral picture is consistent with what Greenblatt [4] found with commercial frontier models on harder tasks: smooth, length-dependent uplift from filler tokens. We additionally find sensitivity to disruptive filler sequences and observe diminishing returns at very long lengths.

## 4 Mechanistic evidence that filler encodes task-relevant information

What do filler positions encode, and how does the model use them? We give three lines of evidence: a question  $\rightarrow$  filler  $\rightarrow$  answer relay (attention), intermediate values encoded at filler positions (logit lens), and causal dependence of the answer on that content (KV-cache transplants). These analyses use local 4-bit quantized versions of DeepSeek V3 and Kimi K2 after verifying that filler uplift survives quantization (Appendices B and C).

### 4.1 Attention

When filler tokens are present, the model shifts attention from the question to the filler tokens, which in turn attend to the question and to one another, such that the question is re-processed repeatedly through the filler region. In both baseline and filler conditions, the answer position devotes  $\sim 70\text{--}90\%$  of its attention to the system prompt and few-shot examples, leaving  $\sim 20\%$  as the routing budget for intermediate computation; this is where filler shifts attention. In baseline, DeepSeek V3’s attention from the answer position to the question peaks at 15.8% at layer 40. With filler, attention to the question drops to 3–4% at the same layer and the freed budget is redirected to the filler region (13% for  $k = 10$ , 18% for  $k = 100$ ). Within the filler, late positions attend to earlier ones at  $\sim 19\text{--}23\%$  at layer 40, and filler positions read from the question at 5–12% in middle layers. The result is a relay of question  $\rightarrow$  filler  $\rightarrow$  answer that replaces the answer’s direct line to the question.

### 4.2 Logit lens reveals encoded intermediates

Filler positions encode task intermediates ( $A_1$ ,  $A_2$ ) directly in their residual stream. We show this by projecting the residual at every (layer, filler position) through the model’s RMSNorm and unembedding, restricting to numeric tokens, and asking: how often is the top numeric token exactly equal to the ground-truth intermediate? Figure 3 shows the answer for 2-fact addition on DeepSeek V3 with  $k = 10$  dot filler.

Across all examples,  $A_1$  is most strongly encoded in the first few filler tokens while  $A_2$  appears later. In correct examples, the sum  $A_1 + A_2$  then appears in the last layers just before the answer position. The no-filler baseline has only the question-end and answer positions, with no filler region between them; there the sum  $A_1 + A_2$  is strongly decoded value at the answer position (for correct examples; Appendix D), while  $A_1$  and  $A_2$  decode weakly and do not appear clearly separated like in filler. In wrong filler examples, the sum is absent while  $A_1$  and  $A_2$  remain encoded: the model fails not by failing to retrieve, but by failing to compose. Decoding the residual stream thus reveals not just what the model thought, but also where its reasoning failed. Figure 3 shows results only for DeepSeek V3 dot filler with  $k = 10$ , but results are similar across longer filler lengths, different filler types, and both models (Appendix D). The picture is also stable under precision and under lens choice: on the native-FP8 DeepSeek-V3-0324 checkpoint,  $A_1$ ,  $A_2$ , and  $A_1 + A_2$  remain recoverable at comparable strength and the same positions, up to a four-position shift of the  $A_2$  peak, and a Jacobian lens [26] fit to DeepSeek V3 [27] picks out the same peak (layer, position) cell for all three values, with accuracies within 2 points of the logit lens (Appendix N).

The same readout also lets us compare how a computation is laid out under filler versus under an explicit chain-of-thought. Figure 4 does this for the system-of-equations task on DeepSeek V3, aggregating 500 examples in each of three conditions: no filler, 25 dots of filler, and explicit written

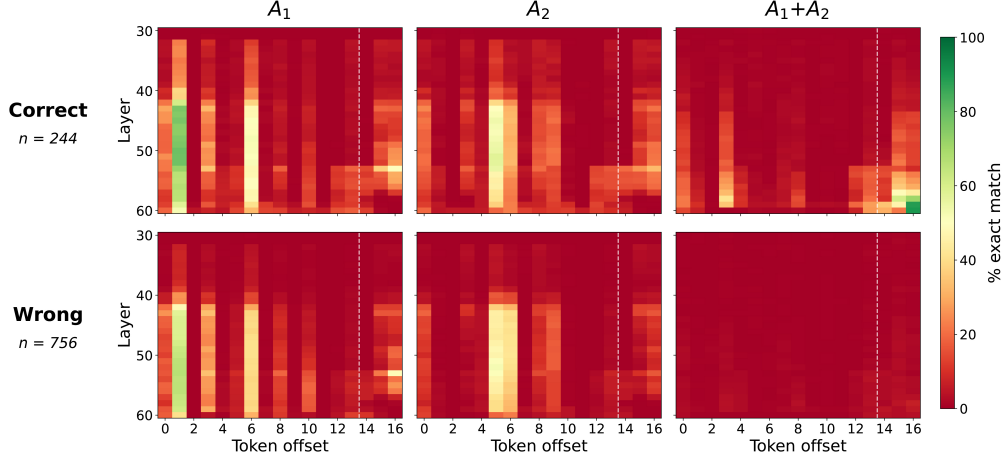


Figure 3: **Logit-lens heatmaps show intermediate values of hidden computation are encoded in the residual stream over filler positions.** For the 2-fact addition task, we apply the logit lens at each (layer, filler position). Color shows the fraction of examples whose top numeric token exactly matches the target ( $A_1$ ,  $A_2$ , or  $A_1+A_2$ ). *Top (correct examples)*:  $A_1$  is encoded early in the filler,  $A_2$  later, and  $A_1+A_2$  right before the answer position. *Bottom (wrong examples)*:  $A_1$  and  $A_2$  are encoded but the sum is absent: the model retrieves the facts but fails to add them. The dashed line marks where the answer is prompted.

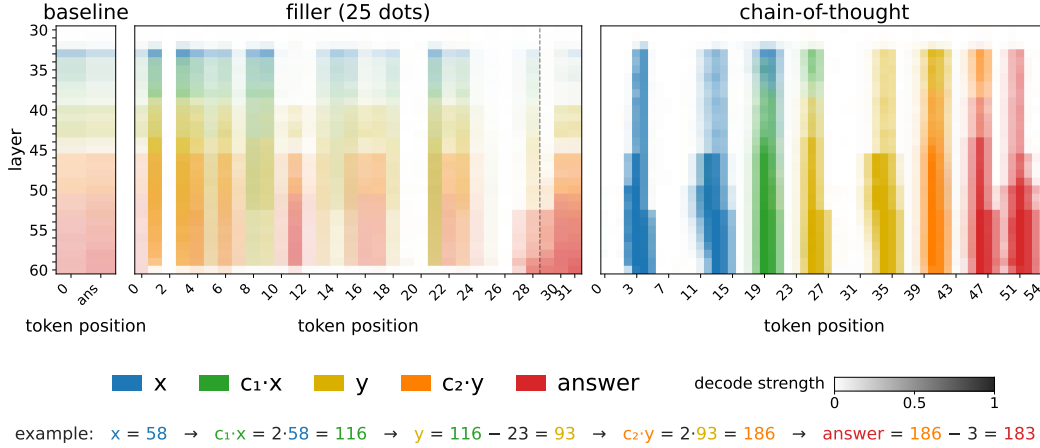


Figure 4: **Logit-lens heatmaps show that filler computation is not the same as chain-of-thought when solving systems of equations.** (DeepSeek V3; each panel aggregates 500 examples). Each chain quantity  $x \rightarrow c_1x \rightarrow y \rightarrow c_2y \rightarrow \text{answer}$  has a fixed color; opacity at a (layer, position) is its average decode strength. *Left* (no filler): the chain resolves in a single pass at the answer position, operands in earlier layers and the answer in later ones. *Middle* (25 dots): the same depth-ordered intermediates spread across the filler positions. *Right* (explicit CoT): each intermediate surfaces right before the position where it is written out.

reasoning (with all few-shot examples showing a consistent reasoning format so that the written steps fall at consistent positions across examples). Under filler, the intermediates of the chain ( $x$ ,  $c_1x$ ,  $y$ ,  $c_2y$ , answer) are encoded across the filler positions in the same depth order in which the no-filler baseline forms them in a single pass at the answer position. This is true across different filler types and lengths. Under explicit chain-of-thought, each intermediate instead surfaces directly before the token position where it is written out. Aggregated across these 500 examples, the filler residual stream resembles the model’s ordinary single-pass computation spread across the available positions more than it resembles a written step-by-step derivation.

### 4.3 KV-cache transplants

The model attends to filler positions and encodes intermediate values there; we now show it causally relies on that content. Our KV-cache transplant is a form of activation patching, a standard tool in mechanistic interpretability for testing whether specific hidden states causally carry information [28–31]. On 1-fact addition, we run the model on 500 pairs of examples that share the addend  $X$  but differ in the fact  $A$ , and transplant the donor’s KV cache *at only the filler-token positions* into the target’s forward pass. For  $k = 100$  dots, the donor’s  $A+X$  answer rank drops from  $96^{+12}_{-15}$  (95% CIs) in the unmodified target to  $11^{+2}_{-3}$  after transplant; the effect holds for dots and counting filler ( $k = 10\text{--}100$ ) and grows with filler length. Full answer swaps (the target outputs the donor’s  $A+X$ ) occur  $13^{+3}_{-2}\%$  of the time for dots  $k = 100$ , rising to  $22^{+10}_{-7}\%$  when the donor was correct and the target incorrect.

The same holds when the filler value is computed rather than retrieved. On the system of equations, we transplant the donor’s filler KV ( $k = 100$  dots) on 409 matched pairs that share the question’s arithmetic but differ in  $y$ , so that adopting the donor’s  $y$  would make the target emit the donor’s answer. The donor-answer rank again drops sharply, from  $236^{+47}_{-35}$  to  $15^{+5}_{-3}$ . The transplant changes the target’s answer  $34.7^{+4.7}_{-4.4}\%$  of the time; in  $6.4^{+2.4}_{-2.2}\%$  of cases the model outputs the donor’s answer outright (vs. 13% for 1-fact). Most changes corrupt the target’s answer rather than fully swap it, but a patch confined to the filler positions that drives the donor’s specific answer up by hundreds of ranks establishes that filler-position content is causal for the final answer.

The 2-fact task tests whether the *positional* structure is itself causal. Since  $A_1$  and  $A_2$  decode at distinct filler positions (Figure 3), we build fact-matched pairs that hold one addend fixed and vary the other. For  $A_1$  ( $n = 405$ ,  $k = 50$  dots), transplanting the whole filler region shifts the donor answer’s rank by  $70^{+15}_{-9}$  places ( $88 \rightarrow 18$ ); transplanting *only* the 12 positions where  $A_1$  is decoded in  $\geq 15\%$  of examples recovers  $65^{+11}_{-11}$  of those ranks (93%), while the 41 complementary positions shift it by just  $12^{+4}_{-3}$  and never swap.  $A_2$  behaves the same (16 positions recover 85% of the shift; Appendix E). Each addend is independently and positionally manipulable, so the positional structure is causal, not a logit-lens artifact.

## 5 Unsupervised decoding pipeline

The heatmaps in Figure 3 require knowing the ground-truth intermediate. A white-box auditor has only the model, the prompt, and the output. We now describe a pipeline (Figure 5) that takes hidden states alone as input and produces a guess about the hidden reasoning, using no labels.

**Stage 1: Hidden-state extraction.** For each example, we run the model and extract the residual stream at every layer and every filler-token position ( $\sim 7000$ -dim vectors at each of 60 layers for DeepSeek V3 and Kimi K2).

**Stage 2: Residual logit lens per (layer, filler position).** We pass each hidden state through the model’s final RMSNorm and unembedding (the logit lens) to obtain a distribution over the  $\sim 129\text{K}$ -token vocabulary, restricting to the second half of the model (layers 30–60 for both models), where logit-lens readouts are more reliable.

We then compute, for each (layer, position), the cross-example mean probability for each token and define the per-example residual as the example’s distribution minus this mean. The residual step generally helps by removing tokens that are always high (e.g., formatting artifacts), though it can suppress useful tokens when the filler itself is numeric (counting filler); we report residualized results by default and ablate the step in Appendix L. We save the top- $T$  tokens by residual score per example per (layer, position) ( $T = 30$  by default).

**Stage 3: Aggregation across filler positions.** Tokens related to the computation should be encoded repeatedly across the filler region, so for each example we sum residual scores across all (layer, position) cells, rank tokens by the aggregated score, and keep the top 50. This pools both singularly strong and weak-but-consistent encodings into one ranked list.

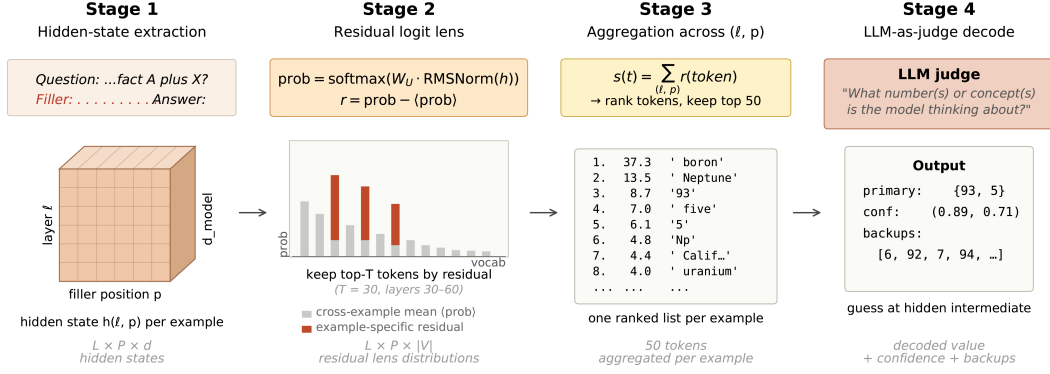


Figure 5: **The unsupervised decoding pipeline.** (1) Extract residual-stream activations at every layer and filler position. (2) Apply the logit lens at each cell and subtract a cross-example mean. (3) Aggregate residual scores across cells; keep the top 50 tokens per example. (4) Pass them to a judge LLM under a neutral prompt. No training or labels are required.

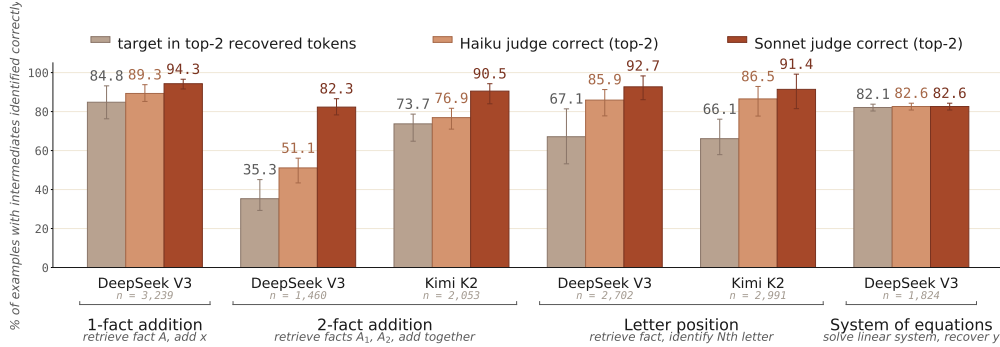


Figure 6: **Decoding accuracy across tasks, models, and judges.** For each task–model pair: (grey) the intermediate(s) are in the top-2 decoded tokens, no judge; (light rust) a *Haiku* judge given the top-50 tokens places them in its top-2 guesses; (rust) the same with *Sonnet*. Bars pool 6–9 filler conditions; error bars are the min/max across conditions. Judges exploit contextual tokens beyond the literal top-2, so they add nothing on the system of equations, whose task involves no context. Shuffled-token baseline accuracy is  $\sim 1\%$  (Appendix H).

**Stage 4: LLM decode.** The top-50 list contains the intermediate(s) but is noisy (punctuation, subword fragments, near-misses). We hand it, with scores, to an LLM judge (Claude Haiku 4.5 or Claude Sonnet 4.6) and ask in a neutral prompt what number(s) or concept(s) the model was reasoning about (Appendix F). The judge returns a primary guess and up to 10 backups, and we score whether the true intermediate is in its top- $K$ . We withhold the original prompt and output from the judge, since they would make its job too easy; in a real auditing setting this could be provided.

We also evaluate without a judge by checking whether the intermediate appears among the aggregated top tokens directly, by exact rank among numeric tokens for numeric intermediates and by a multilingual top- $K$  substring measure for string intermediates (Appendix G).

## 6 Decoding results

For each task–model combination, we report three numbers (Figure 6): the fraction of examples for which the ground-truth intermediate is in the top-2 decoded tokens after aggregation (no judge), the fraction for which a *Haiku* judge includes the target among its top-2 guesses after seeing the top-50 decoded tokens, and the same for a *Sonnet* judge. We report accuracies pooled across filler conditions (per-condition variation is within  $\pm 5$  points; Appendix H), on correct examples, which have a well-

defined target. On incorrect examples, the decoder reveals whether retrieval or composition failed (Appendix J). Across all tasks, decoding “failures” are structured rather than noise. The top tokens are typically another quantity from the same computation (the prompt addend, the sum, an adjacent step of the chain) or a  $\pm 1$  near miss (Appendix K).

### 6.1 A single retrieved intermediate: 1-fact addition on DeepSeek V3

For 1-fact addition, the intermediate is the retrieved fact  $A$ . Pooling nine filler conditions ( $n = 3,239$ ; dots, alphabet, and counting,  $k = 5\text{--}100$ ), the target is in the top-2 numeric tokens in 84.8% of examples with no judge; Haiku raises this to 89.3% and Sonnet to 94.3%. Every filler condition shows the same pattern.

### 6.2 Two parallel intermediates: 2-fact addition

The 2-fact task is the most demanding: the model must retrieve two values and combine them. Pooling DeepSeek V3 results across six filler conditions ( $n = 1,460$ ), both  $A_1$  and  $A_2$  appear in the top-2 numeric tokens in 35.2% of examples. The Haiku judge raises identification of both targets to 51.1%, and Sonnet to 82.3%. On Kimi K2 ( $n = 2,082$ ), the corresponding numbers are 74.1% / 75.7% / 90.0%. Kimi K2 encodes the intermediates more cleanly.

The judge gap is much larger here than on 1-fact because with two intermediates the top-2 numeric tokens often miss one. In DeepSeek V3 dots-25,  $A_1$  is in the top-2 in 80% of examples but  $A_2$  in only 48%. The judge sees the full top-50 list and can use context (word tokens, two numbers that add to a third) to recover both.

The pipeline also works reasoning-trained Kimi K2.5. On 2-fact addition both addends are in the top-2 numeric tokens in 80.6% of examples ( $n = 962$ , dots-10/25), with the same residual-stream structure as Kimi K2 (Appendix O). Decoding is also stable under precision: on DeepSeek V3 dots-10, direct top-10 recovery is 63.1% (4-bit AWQ) vs. 63.6% (native FP8), and Sonnet top-2 is 82.4% vs. 86.6% (Appendix N).

**What the judge adds.** Does the judge recover information present in the list or infer it from task priors? We ran two controls (Appendix M). Stripping every non-numeric token from the top-50 list drops Sonnet’s both-addends accuracy to the direct-token rate: 82.3%  $\rightarrow$  36.4% on DeepSeek V3 (direct: 35.2%) and 90.0%  $\rightarrow$  73.1% on Kimi K2 (direct: 74.1%). Keeping all tokens but replacing  $A_1$  and  $A_2$  with random integers in  $[1, 100]$  collapses accuracy to 39.2% and 28.2%. The judge needs both the addends and the context tokens (element names and fragments): it selects among candidates in the list rather than generating them, and on the system of equations (Section 6.4), which offers no context, it adds nothing. Judge-assisted rates are thus a context-assisted metric.

### 6.3 Non-numeric intermediates: letter-position tasks

The pipeline generalizes to multi-token word intermediates: here, a city or element name. For direct scoring, we check whether the target appears among the top-2 decoded string tokens, allowing English or Chinese matches (both models frequently “think” in Chinese [32, 33]; Appendix G). On DeepSeek V3 ( $n = 2,702$ ), a match for the target appears in the top-2 string tokens for 66.9% of examples, Haiku reaches 85.9%, and Sonnet reaches 92.7%. On Kimi K2 ( $n = 2,997$ ), the corresponding numbers are 62.4% / 83.9% / 90.2%. A judge prompt that names the target type helps where there is headroom, by up to 11–13 points on the harder city subtask (Appendix F).

### 6.4 In-context computation: system of equations

The system-of-equations task tests whether the pipeline also recovers a value computed in context. The value  $y$  sits mid-chain in  $x \rightarrow c_1x \rightarrow y \rightarrow c_2y \rightarrow \text{answer}$ , where only  $x$  and constants are given in the prompt. We score recovery of  $y$ , the one intermediate that is a named variable. Pooling DeepSeek V3 across six filler conditions ( $n = 1,824$ ),  $y$  appears in the top-2 numeric tokens in 82.1% of examples, and the judges add essentially nothing (Haiku 82.6%, Sonnet 82.6%). This is because the prompt is built from nonsense variable names and random numbers, so there is no context for a judge to exploit. The other hidden steps decode at varying rates, and within the top-10 tokens every step from  $x$  to the answer is found 51–97% of the time (Appendix I).

## 7 Limitations

**Scope of the claims.** Our tasks are purposefully selected to have a discrete, nameable intermediate (a number, an element, a city) such that ground truth can be known. They are also non-adversarial. The monitorability claims are made for these conditions and no other. Our claim is only that hidden filler computation is at least partially accessible on cleanly decomposable tasks.

**What the readout can see.** The pipeline reads residual content through the unembedding matrix, so computation in attention patterns or in directions orthogonal to the vocabulary projection is invisible to it. Changing the lens would not lift this restriction. A Jacobian lens reproduces our readout (Section 4), and a tuned lens [21] would, if anything, bias it toward the final output since its probes are fit to the final-layer distribution rather than to mid-computation values. Intermediates that are not a single tokenizable concept, such as a goal or an intent to deceive, are harder but not necessarily out of reach – a capable judge could infer them from related tokens in the top- $K$  list. The harder cases are concepts whose vocabulary never surfaces, non-tokenizable intermediates such as distributions, strategically chosen fillers, and computation optimized to look like benign token activity. Whether residual-stream decoding extends to these, perhaps via patchscopes [23] or trained probes, is open.

**Fidelity of the readout.** The logit lens is a coarse readout, unreliable in early layers and on tokenizer artifacts [20]. To mitigate this, we restrict to the second half of layers and aggregate across positions. The mechanistic and decoding analyses use 4-bit checkpoints. Uplift survives quantization, the KV-cache results are lens-independent, and the DeepSeek V3 2-fact analysis is unchanged on the native-FP8 checkpoint (Appendix N), but we have not repeated every analysis at higher precision. The filler-versus-CoT comparison (Figure 4) is a visual aggregate on a single task, not a quantitative claim that the two are algorithmically distinct.

**Model coverage.** Our hidden-state analyses cover two instruction-tuned models in depth and one reasoning-trained model (Kimi K2.5, with explicit reasoning suppressed) on one task; whether RL training for chain-of-thought changes what a model does latently when it is allowed to reason explicitly remains open. The pipeline requires full hidden states, so it cannot be verified on closed models such as Claude, Gemini, or GPT [4].

## 8 Discussion

The combined behavioral, mechanistic, and decoding evidence shows that filler positions read information from the question, hold and process intermediate values across the residual stream, and are read out by the answer position. For 2-fact tasks, the residual stream encodes  $A_1$  early in the filler,  $A_2$  later, and the sum in late layers just before the answer position. By reading the residual stream, we are also able to identify where the reasoning fails when the model answers incorrectly.

This has implications for CoT monitorability. Filler-token computation is a limit case where reasoning is performed in tokens whose surface form is uninformative [2, 3], and indeed as Sections 3 and 4 confirm, filler computation is real and not visible in the output text. But behavioral monitoring is not the only auditing tool available. Cheap interpretability tools (logit lens with cross-example normalization and an LLM judge) can recover most of the hidden intermediates on our tasks without supervision. The implication is not that CoT monitoring is safe, but that the gap is at least partially closable by reading the residual stream. Interpretability monitoring is a meaningful complement to behavioral monitoring.

Several open questions follow from the picture above. First, at the circuit level: which attention heads implement the question  $\rightarrow$  filler  $\rightarrow$  answer relay, and do they overlap with heads used in explicit-CoT problem solving? Second, on harder intermediates: how far can the pipeline be extended to tasks where the relevant computation is not a single nameable token, possibly via trained probes, patchscope-style elicitation, or natural language autoencoders? Third, on adversarial robustness: can a model trained to use extra tokens evasively, e.g., to distribute its computation across the residual stream, still be decoded by a more sophisticated auditor?

Overall, pessimism about CoT monitorability is incomplete. Hidden computation, a concrete case where behavioral oversight fails, is at least partially accessible to interpretability on cleanly decomposable tasks. Opaque is not necessarily the same as unauditable.

## Acknowledgments and Disclosure of Funding

K.B. is supported by the Cambridge Boston Alignment Initiative, which funded this work. In particular, we thank Emre Yavuz for his support.

## References

- [1] Tomek Korbak, Mikita Balesni, Elizabeth Barnes, Yoshua Bengio, Joe Benton, Joseph Bloom, Mark Chen, Alan Cooney, Allan Dafoe, Anca D. Dragan, Scott Emmons, Owain Evans, David Farhi, Ryan Greenblatt, Dan Hendrycks, Marius Hobbhahn, Evan Hubinger, Geoffrey Irving, Erik Jenner, Daniel Kokotajlo, Victoria Krakovna, Shane Legg, David Lindner, David Luan, Aleksander Madry, Julian Michael, Neel Nanda, Dave Orr, Jakub Pachocki, Ethan Perez, Mary Phuong, Fabien Roger, Joshua Saxe, Buck Shlegeris, Martín Soto, Eric Steinberger, Jasmine Wang, Wojciech Zaremba, Bowen Baker, Rohin Shah, and Vladimir Mikulik. Chain of thought monitorability: A new and fragile opportunity for ai safety. *CoRR*, abs/2507.11473, July 2025. URL <https://doi.org/10.48550/arXiv.2507.11473>.
- [2] Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamile Lukosiute, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, Saurav Kadavath, Shannon Yang, Thomas Henighan, Timothy Maxwell, Timothy Telleen-Lawton, Tristan Hume, Zac Hatfield-Dodds, Jared Kaplan, Jan Brauner, Samuel R. Bowman, and Ethan Perez. Measuring faithfulness in chain-of-thought reasoning. *CoRR*, abs/2307.13702, 2023. URL <https://doi.org/10.48550/arXiv.2307.13702>.
- [3] Jacob Pfau, William Merrill, and Samuel R. Bowman. Let’s think dot by dot: Hidden computation in transformer language models. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=NikbrdtYvG>.
- [4] Ryan Greenblatt. Recent llms can use filler tokens or problem repeats to improve (no-cot) math performance. <https://www.lesswrong.com/posts/NYzYJ2WoB74E6uj9L>, 2025. LessWrong / AI Alignment Forum.
- [5] Ryan Greenblatt. Recent LLMs can do 2-hop and 3-hop latent (no-CoT) reasoning on natural facts. <https://www.lesswrong.com/posts/aYtrLhoZtCKZnfBvA>, 2026. LessWrong / AI Alignment Forum.
- [6] Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. *Advances in Neural Information Processing Systems*, 36:74952–74965, 2023.
- [7] Iván Arcuschin, Jett Janiak, Robert Krzyzanowski, Senthoooran Rajamanoharan, Neel Nanda, and Arthur Conmy. Chain-of-thought reasoning in the wild is not always faithful. *arXiv preprint arXiv:2503.08679*, 2025.
- [8] Yanda Chen, Joe Benton, Ansh Radhakrishnan, Jonathan Uesato, Carson Denison, John Schulman, Arushi Somani, Peter Hase, Misha Wagner, Fabien Roger, et al. Reasoning models don’t always say what they think. *arXiv preprint arXiv:2505.05410*, 2025.
- [9] Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. *arXiv preprint arXiv:2503.11926*, 2025.
- [10] Scott Emmons, Erik Jenner, David K Elson, Rif A Saurous, Senthoooran Rajamanoharan, Heng Chen, Irhum Shafkat, and Rohin Shah. When chain of thought is necessary, language models struggle to evade monitors. *arXiv preprint arXiv:2507.05246*, 2025.
- [11] Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=ph04CRkPdC>.
- [12] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=NG7sS51zVF>.

- [13] Sohee Yang, Elena Gribovskaya, Nora Kassner, Mor Geva, and Sebastian Riedel. Do large language models latently perform multi-hop reasoning? In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10210–10229, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.550. URL <https://aclanthology.org/2024.acl-long.550/>.
- [14] Sohee Yang, Nora Kassner, Elena Gribovskaya, Sebastian Riedel, and Mor Geva. Do large language models perform latent multi-hop reasoning without exploiting shortcuts? In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 3971–3992, 2025.
- [15] Eden Biran, Daniela Gottesman, Sohee Yang, Mor Geva, and Amir Globerson. Hopping too late: Exploring the limitations of large language models on multi-hop queries. *CoRR*, abs/2406.12775, 2024. URL <https://doi.org/10.48550/arXiv.2406.12775>.
- [16] Fangwei Zhu, Peiyi Wang, and Zhifang Sui. Chain-of-thought tokens are computer program variables. *arXiv preprint arXiv:2505.04955*, 2025.
- [17] Yixiao Huang, Hanlin Zhu, Zixuan Wang, Jiantao Jiao, Stuart Russell, Somayeh Sojoudi, and Song Mei. Transformers provably learn to internalize chain-of-thought. *arXiv preprint arXiv:2605.28600*, 2026.
- [18] Uri Katz, Mor Geva, and Jonathan Berant. Inferring implicit relations in complex questions with language models. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 2548–2566. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.findings-emnlp.188.
- [19] Anum Afzal, Florian Matthes, Gal Chechik, and Yftah Ziser. Knowing before saying: LLM representations encode information about chain-of-thought success before completion. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 12791–12806. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-acl.662.
- [20] nostalgebraist. Interpreting GPT: The logit lens. <https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>, 2020. LessWrong / AI Alignment Forum.
- [21] Nora Belrose, Zach Furman, Logan Smith, Danny Halawi, Igor Ostrovsky, Lev McKinney, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the tuned lens. *CoRR*, abs/2303.08112, 2023. URL <https://doi.org/10.48550/arXiv.2303.08112>.
- [22] Koyena Pal, Jiuding Sun, Andrew Yuan, Byron Wallace, and David Bau. Future lens: Anticipating subsequent tokens from a single hidden state. In *Proceedings of the 27th Conference on Computational Natural Language Learning (CoNLL)*, page 548–560. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.conll-1.37. URL <http://dx.doi.org/10.18653/v1/2023.conll-1.37>.
- [23] Asma Ghandeharioun, Avi Caciularu, Adam Pearce, Lucas Dixon, and Mor Geva. Patchscopes: A unifying framework for inspecting hidden representations of language models. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=5uwBzcn885>.
- [24] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.
- [25] Moonshot AI. Kimi k2.5: Visual agentic intelligence. <https://huggingface.co/moonshotai/Kimi-k2.5>, 2026. Model card and technical blog; released January 2026.
- [26] Wes Gurnee, Nicholas Sofroniew, Adam Pearce, Mateusz Piotrowski, Isaac Kauvar, Runjin Chen, Anna Soligo, Paul Bogdan, Euan Ong, Rowan Wang, Ben Thompson, David Abrahams, Subhash Kantamneni, Emmanuel Ameisen, Joshua Batson, and Jack Lindsey. Verbalizable representations form a global workspace in language models. *Transformer Circuits Thread*, 2026. URL <https://transformer-circuits.pub/2026/workspace/index.html>.
- [27] Kaley Brauer. J-lens shouldn’t target the final layer by default. LessWrong, 2026. URL <https://www.lesswrong.com/posts/Adb55vLqt33LEyBfz/j-lens-shouldn-t-target-the-final-layer-by-default>. Lens and code: <https://huggingface.co/kbrauer/deepseek-v3-jacobian-lens>.

- [28] Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- [29] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022.
- [30] Kevin Ro Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small. In *The Eleventh International Conference on Learning Representations*, 2023.
- [31] Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching. *arXiv preprint arXiv:2404.15255*, 2024.
- [32] Yihao Li, Jiayi Xin, Miranda Muqing Miao, Qi Long, and Lyle Ungar. The impact of language mixing on bilingual llm reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 32519–32536, 2025.
- [33] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL <http://dx.doi.org/10.1038/s41586-025-09422-z>.
- [34] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. In P. Gibbons, G. Pekhimenko, and C. De Sa, editors, *Proceedings of Machine Learning and Systems*, volume 6, pages 87–100, 2024. URL [https://proceedings.mlsys.org/paper\\_files/paper/2024/file/42a452cbafa9dd64e9ba4aa95cc1ef21-Paper-Conference.pdf](https://proceedings.mlsys.org/paper_files/paper/2024/file/42a452cbafa9dd64e9ba4aa95cc1ef21-Paper-Conference.pdf).
- [35] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang,

- Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, and Wangding Zeng. Deepseek-v3 technical report. *CoRR*, abs/2412.19437, 2024. URL <https://doi.org/10.48550/arXiv.2412.19437>.
- [36] Kimi Team, Yifan Bai, Yiping Bao, Y. Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Chenxiao Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Qizheng Gu, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yang Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Haoyu Lu, Lijun Lu, Yashuo Luo, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Zeyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Lin Sui, Xinjie Sun, Flood Sung, Yunpeng Tai, Heyi Tang, Jiawen Tao, Qifeng Teng, Chaoran Tian, Chensi Wang, Dinglu Wang, Feng Wang, Hailong Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Si Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Haoning Wu, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Jing Xu, Jing Xu, Junjie Yan, Yuzi Yan, Hao Yang, Xiaofei Yang, Yi Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Siyu Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Shaojie Zheng, Longguang Zhong, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Zhen Zhu, Weiyu Zhuang, and Xinxing Zu. Kimi K2: Open Agentic Intelligence. *arXiv e-prints*, art. arXiv:2507.20534, July 2025. doi: 10.48550/arXiv.2507.20534.

## Supplementary Material

*Supplementary material for the paper “Reading Between the Dots: Decoding Hidden Computation across Filler Tokens.”*

This appendix collects experimental details and supplementary results that did not fit in the main text. In Appendix A, we describe the four task families in detail, including fact sources, filtering and few-shot construction, the conditions evaluated, and full example prompts. In Appendix B, we document the specific 4-bit quantized checkpoints of DeepSeek-V3 and Kimi K2 used throughout the paper, including which layers and projections are held at higher precision. In Appendix C, we report the computational requirements of every experiment and pipeline stage (model loading, hidden-state extraction, KV-cache transplants, attention analysis, residual decoding, and LLM-as-judge calls) together with hardware and storage budgets. In Appendix D, we provide baseline and additional logit-lens heatmaps showing that the encoding pattern observed for DeepSeek V3 with dot-10 filler in Figure 3 generalizes to Kimi K2, to counting filler, and to longer filler lengths. In Appendix E, we report full results for the KV-cache transplants with 2-fact addition. In Appendix F, we give the full prompts shown to the LLM judges (Claude Haiku and Claude Sonnet), in both the neutral and task-specific framings discussed in Section 6. In Appendix G, we define the multilingual top- $K$  substring-coverage measure used to score string-valued intermediates without an LLM judge. In Appendix H, we report the full top- $K$  decoding results for all tasks, models, filler conditions, and judges. We also provide shuffled-token baseline controls that show judge performance is not merely prompt-induced guessing. Appendix I gives the full per-intermediate decoding breakdown for the system-of-equations task. In Appendix J, we provide the decoding results for incorrect examples and investigate failure modes. Appendix K analyzes the structure of decoding failures on each task. In Appendix L we ablate the residual-subtraction step used in the main decoding pipeline and show that it can be significantly beneficial but not always. Appendix M reports the judge controls (numbers-only lists and lists with the true addends removed) that bound how much the LLM judge recovers from the token list versus infers. Appendix N repeats the DeepSeek V3 2-fact logit-lens and decoding analysis on the native-FP8 checkpoint and compares the logit lens with a Jacobian lens. Finally, Appendix O reports the behavioral, model-generated-filler, and decoding results on the reasoning-trained Kimi K2.5.

### A Task information and example prompts

#### 1-fact addition

*Types of facts:* Facts are drawn from Ryan Greenblatt’s `compose_facts` repository.<sup>2</sup>

- 125 age facts (age at which famous person died; range 20–97);
- 118 atomic number facts (range 1–118);
- 76 static facts (generic facts like vertebrae count, opera counts, alphabet sizes; range 6–600)

*Fact filtering and few-shot:* When creating the 1-fact addition dataset, we first check that the model (DeepSeek V3 or Kimi K2) knows the answer when asked as a standalone question. Any fact not answered correctly at least 3/4 times is filtered out of the dataset. Then, 5 facts are randomly excluded from the dataset to serve as few-shot examples. Each prompt uses the same 5 few-shot examples.

*Task format:* What is <fact A> plus <random 2-digit addend X>?

*Conditions:* dot filler with  $k = 10, 25, 50$ ; counting filler with  $k = 5, 10, 25$ ; alphabet filler with  $k = 10, 25, 50$

*Example prompt:* For dots,  $k = 10$ . The system prompt is slightly varied for each condition to reflect the correct filler type and length.

### system

<sup>2</sup>[https://github.com/rgreenblatt/compose\\_facts](https://github.com/rgreenblatt/compose_facts)

You will be given a question. Answer immediately using the format 'Answer: [ANSWER]' where [ANSWER] is just the number, nothing else. No explanation, no words, no reasoning, just the number. After the question, there will be 10 filler tokens (a sequence of dots) before you answer.

### user

Question: What is the age at which Søren Kierkegaard died plus 96?

Filler: . . . . .

Answer:

### assistant

138

### user

Question: What is the age at which Elizabeth I of England died plus 40?

Filler: . . . . .

Answer:

### assistant

109

[... 3 more few-shot pairs (Coltrane / oxygen / phosphorus) ...]

### user

Question: What is the atomic number of silicon plus 62?

Filler: . . . . .

Answer:

## 2-fact addition

*Types of facts:* 100 atomic number facts (range 1–100)

*Fact filtering and few-shot:* Because this task is much more difficult than 1-fact addition, we restrict to only atomic number facts  $\leq 100$ . 10 facts are randomly excluded from the dataset to serve as few-shot examples. Each prompt uses the same 5 few-shot examples.

*Task format:* What is the atomic number of <element A1> plus the atomic number of <element A2>?

*Conditions:* dot filler with  $k = 10, 25, 50$ ; counting filler with  $k = 10, 25, 50$

*Example prompt:* Same format as 1-fact addition.

## 2-hop letter position

*Types of facts:*

- 367 capitals of countries/states/provinces/territories
  - includes all countries, US states, Chinese provinces/autonomous regions, Indian states, Brazilian states, German states, Canadian provinces/territories, Mexican states, and Australian states/territories that do not share all or most of their name with their capital city
- 100 chemical element facts

- includes all chemical elements from 2-fact addition dataset, inverted to ask for the element name instead of atomic number

*Fact filtering and few-shot:* 5 facts are randomly excluded from the capitals dataset and from the chemical elements dataset to serve as few-shot examples. Each prompt uses the same 5 few-shot examples.

*Task format:* What is the <second/third/last> letter of the chemical element with atomic number <A>; What is the last letter of the name of the capital of <country/state/province/territory>?

*Conditions:* dot filler with  $k = 10, 25, 50$ ; counting filler with  $k = 10, 25, 50$

*Example prompt:* Same format as 1-fact addition, except with system prompt modified to say letter instead of number:

```
### system

You will be given a question asking for a specific letter. Answer
immediately with just the single lowercase letter, nothing else. No
explanation, no words, no reasoning, just the letter. After the question,
there will be 10 filler tokens (a sequence of dots) before you answer.

[5 few-shot examples ...]

### user

Question: What is the third letter of the chemical element with atomic
number 47?

Filler: . . . . .

Answer:
```

## System of equations

*Motivation:* The other three tasks use a retrieved fact as the intermediate. The system-of-equations task instead makes every intermediate a value computed from numbers given in the prompt, with nothing to retrieve.

*Task structure:* Each prompt assigns a few nonsense variables literal two-digit values (0–99) and defines others as linear functions of an earlier variable (multiply by a constant of 2 or 3, then add or subtract a constant from 0–50). The question asks for a linear function of one variable. Distractor variables that do not feed the queried variable are included, so the model must bind the correct names rather than the most recent numbers. We write the queried variable’s value as  $y$ , reached by the chain  $x \rightarrow c_1x \rightarrow y \rightarrow c_2y \rightarrow \text{answer}$ , where  $x$  is the in-context value it depends on.

*Conditions:* dot filler with  $k = 10, 25, 50$ ; counting filler with  $k = 5, 10, 25$ .

*Example prompt* (dots,  $k = 10$ ; queried variable cad, giving  $x=\text{bec}=50 \rightarrow c_1x=100 \rightarrow y=\text{cad}=79 \rightarrow c_2y=158 \rightarrow \text{answer}=129$ ):

```
### system

You will be given a list of variable definitions and a question. Answer
immediately with just the number, nothing else. No explanation, no words,
no reasoning, just the number. After the question, there will be 10 filler
tokens (a sequence of dots) before you answer.

[5 few-shot examples ...]

### user

aba = 93
bec = 50
```

```

dab = three times the number for bec plus 20
cad = two times the number for bec minus 21
bah = three times the number for cad minus 44
Question: What is two times the number for cad minus 29?

Filler: . . . . .

Answer:

```

*Easier variation for Kimi K2:* At this task, DeepSeek V3 has significantly higher base accuracy than Kimi K2. To create headroom for measuring behavioral uplift in Kimi K2 (Figure 2), we made the task slightly easier in two ways: we fixed every coefficient to 2 (rather than drawing from  $\{2, 3\}$ ) and drew the additive/subtractive constants from 1–30 (rather than 1–50). All other parameters (five variables, a single chained reference, and literal values in 10–99) were unchanged.

## B Models

We evaluate two pretrained instruction-tuned models. They are run from publicly-available 4-bit quantized checkpoints to fit available GPU memory.

### DeepSeek-V3 (4-bit AWQ).

We use `cognitivecomputations/DeepSeek-V3-0324-AWQ`, a 4-bit AWQ [34] quantization of DeepSeek-V3-0324 [35]. The base model is a 671B-parameter Mixture-of-Experts model with  $\sim 37$ B parameters activated per token. AWQ is applied with group size 128, asymmetric (zero-point) quantization, and the GEMM kernel; the multi-query attention KV projection (`self_attn.kv_a_proj_with_mqa`) is held at FP16.

### Kimi K2 (W4A16).

We use Red Hat / Neural Magic’s `Kimi-K2-Instruct-quantized.w4a16`, a 4-bit-weight / FP16-activation quantization of Moonshot AI’s Kimi-K2-Instruct [36]. The base model is a 1T-parameter Mixture-of-Experts model with  $\sim 32$ B parameters activated per token. Quantization (compressed-tensors format) targets only the routed-expert FFN linears: weights are int4 with symmetric per-group quantization (group size 128), and `lm_head`, all self-attention projections, the shared experts, and the MLP gateup/down projections are held at FP16.

## C Computational requirements

All experiments are run on a single workstation with 3–4 NVIDIA H200 (150 GB HBM3) GPUs.

Full code and aggregated results are available at <https://github.com/kaleybrauer/filler-token-reasoning>. To make the analysis reproducible without requiring readers to rerun the full decoding pipeline, we release the main intermediate and summary outputs used in the paper: per-example aggregated top-50 residual tokens, Claude judge responses, top- $K$  accuracy tables, and logit-lens heatmap summaries. These artifacts cover the correct-example subset across the filler conditions, judges, prompts, and model.

**Model loading.** DeepSeek V3 (4-bit AWQ) is loaded across 3 H200s with  $\sim 110$ – $130$  GB allocated per GPU. Kimi K2 (4-bit W4A16) is loaded via vLLM with tensor parallelism degree 4 on 4 H200s.

**Hidden-state extraction.** For each model and each condition, we run forward passes with hidden states cached at every transformer layer and every prompt-relative token offset, then write per-example pickles to disk. Per example, per condition:  $61 \text{ layers} \times \sim 10 - 100 \text{ positions} \times \text{float16 hidden state}$  ( $d_{\text{model}} = 7168$ ) per example  $\approx 50$  MB / example,  $\approx 50$  GB / condition. Cumulative extraction cache for all examples across all tasks for both models is  $\sim 1$  TB.

**KV-cache transplant.** For each donor/target pair, we run prefill twice, splice the donor’s filler-region key-value cache into the target’s KV state at every layer, and decode the next token. We use 500 random pairs per filler condition on the 1-fact addition task. The intervention runs with quantized DeepSeek V3 on the same 3-H200 setup as extraction.

**Attention analysis.** Filler-attention summaries aggregate per-head attention weights at every (layer, source-position, target-position) intersection on 100 examples per condition for the 1-fact addition task and a no-filler baseline. Per-example attention matrices are reduced on the fly into the segment-level breakdown (system / few-shot / question / filler / answer-label) reported in the paper.

**Residual decode.** Residual fingerprints (per-(example, layer, position) top-50 token IDs after subtracting the cross-example mean softmax distribution) are computed offline by streaming through the cached hidden states with the same RMSNorm + lm\_head used at inference. This step can be run on CPU and runs in tens of minutes per condition.

**LLM-as-judge decoding.** Top-50 residual tokens are scored by Claude Haiku 4.5 and Claude Sonnet 4.6 via the Anthropic API. We issue  $\sim 6$  (conditions)  $\times \sim 250$ – $360$  (examples)  $\times 2$  (judge models)  $\times 2$  (prompt framings: neutral and task-specific) calls per task, totaling  $\mathcal{O}(10^4)$  API calls per task across DeepSeek V3 and Kimi K2.

**Storage.** Quantized model weights occupy  $\sim 840$  GB on local SSD; intermediate extraction caches occupy  $\sim 1$  TB; final aggregated residual JSONs and judge outputs occupy  $< 5$  GB.

## D Logit-lens heatmaps

Figure 3 shows that, for 2-fact addition with DeepSeek V3 and dot-10 filler, the intermediate computation values  $A_1$  and  $A_2$  are encoded in filler token positions. We now show the baseline (no filler) for 2-fact addition (Figure 7). We also show that the same pattern of encoding extends to Kimi K2 (figure 8), counting filler (figure 9), and longer filler lengths (figure 10).

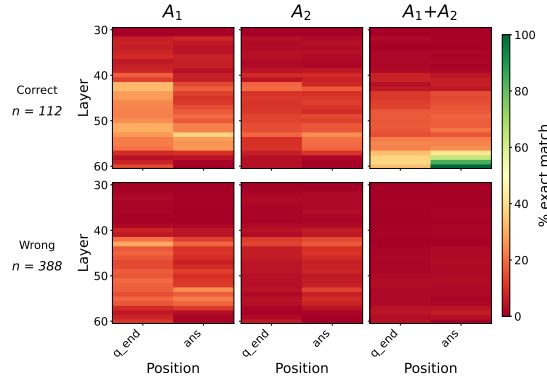


Figure 7: No-filler baseline, 2-fact addition (DeepSeek V3). We apply the logit lens at each (layer, position) for the end of the question and the answer forming position; color shows the fraction of examples whose top numeric token exactly matches the ground-truth target ( $A_1$ ,  $A_2$ , or  $A_1 + A_2$ ), the same measure as Figure 3. Rows split examples by whether the model answered correctly ( $n = 112$  correct,  $n = 388$  wrong). In correct examples (top), the sum  $A_1 + A_2$  is the most strongly decoded value at ans, while  $A_1$  and  $A_2$  decode more weakly there; with no filler region present, the operands do not appear in the separated early/late arrangement they take on under filler (Figure 3). In wrong examples (bottom), the operands remain somewhat encoded while the sum is absent, matching the compose-failure pattern seen under filler.

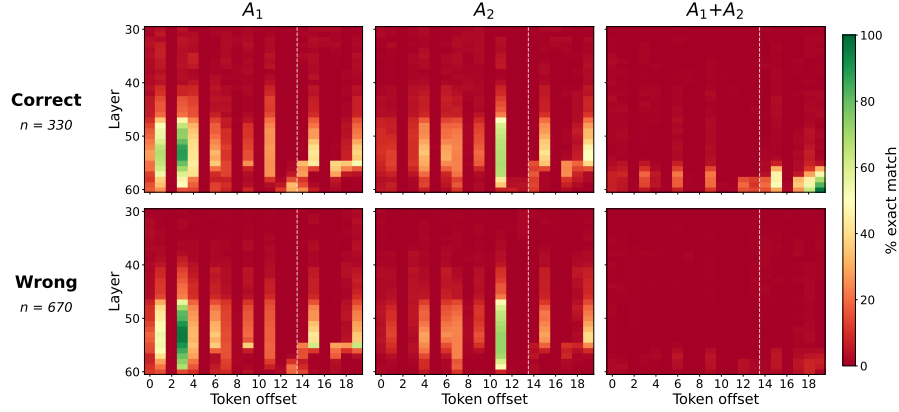


Figure 8: Kimi K2, 2-fact addition with dot-10 filler. We apply the logit lens at each (layer, filler position). Heatmap color shows the fraction of examples where the top number token exactly matches the ground-truth target ( $A_1$ ,  $A_2$ , or  $A_1+A_2$ ). Rows split examples by whether the model answered correctly. The same general pattern is observed as in Figure 3, even with a different model.

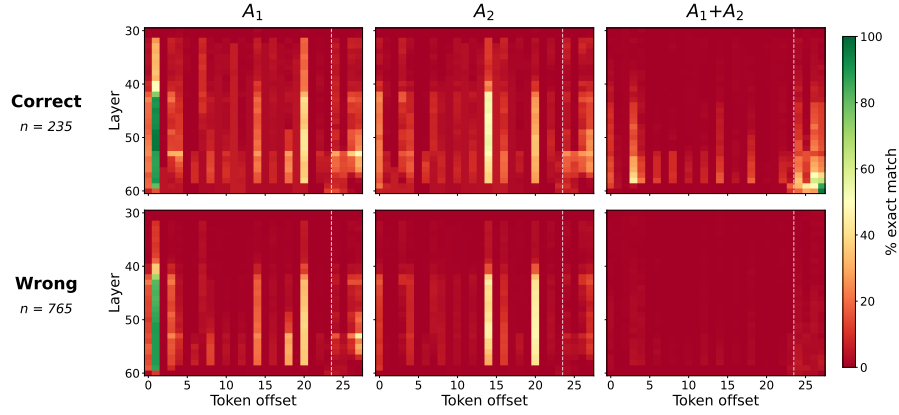


Figure 9: DeepSeek V3, 2-fact addition with counting-10 filler. We apply the logit lens at each (layer, filler position). Heatmap color shows the fraction of examples where the top number token exactly matches the ground-truth target ( $A_1$ ,  $A_2$ , or  $A_1+A_2$ ). Rows split examples by whether the model answered correctly. The same general pattern is observed as in Figure 3, even with a different filler type.

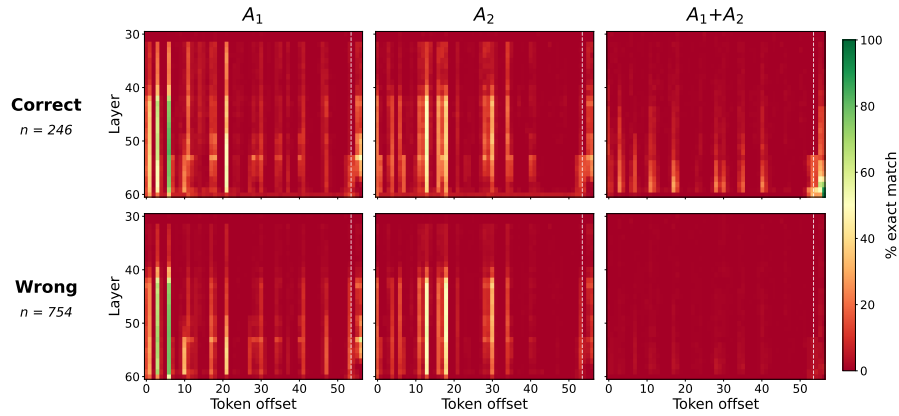


Figure 10: DeepSeek V3, 2-fact addition with dots-50 filler. We apply the logit lens at each (layer, filler position). Heatmap color shows the fraction of examples where the top number token exactly matches the ground-truth target ( $A_1$ ,  $A_2$ , or  $A_1+A_2$ ). Rows split examples by whether the model answered correctly. The same general pattern is observed as in Figure 3, even with much longer filler.

## E Position-resolved KV-cache transplants on 2-fact addition

The main-text transplants (Section 4) establish that filler content is causal for the answer on 1-fact addition and on the system of equations, in both cases by transplanting the entire filler region. This appendix reports the position-resolved 2-fact transplant, which tests the stronger claim that the *positional* structure of the encoding ( $A_1$  and  $A_2$  most strongly decoded at distinct filler positions, as seen in Appendix D) is itself causal. All results are on DeepSeek V3 with  $k = 50$  dot filler (53 filler positions), chosen so that each addend occupies enough distinct positions to transplant separately.

**Position labeling.** Using the logit lens on correct examples, we label a filler position “ $A_1$ ” if, at one or more layers, the top numeric token equals  $A_1$  in more than 15% of examples (threshold  $\theta = 0.15$ ), and analogously for  $A_2$ . At  $k = 50$  this yields 12  $A_1$ -decoded positions and 16  $A_2$ -decoded positions. The two labels are independent: a position may pass both thresholds (for instance, a position decoding  $A_1$  in 50% and  $A_2$  in 35% of examples), in which case it appears in both decoded sets. This is allowed under fact-matching because in the  $A_1$  experiment the  $A_2$  content at such a position is held fixed across donor and target, so only its  $A_1$  content is effectively transplanted (and vice versa). We also ran  $\theta \in \{0.20, 0.30\}$ ; higher thresholds admit fewer positions and weaken the localization, consistent with each addend being distributed across several positions, so we report the most inclusive setting,  $\theta = 0.15$ .

**Fact-matched pairs.** To isolate one addend’s channel, we construct donor/target pairs that share the other addend. For the  $A_1$  channel, donor and target ask for  $A_1^d + A_2$  and  $A_1^t + A_2$  respectively; that is, the same second element  $A_2$ , different first elements ( $A_1^d \neq A_1^t$ ). Because atomic numbers are unique, the held-fixed addend never collides in value. Any shift in the target’s answer is then attributable to the varied addend alone, and the positions encoding the held-fixed addend carry near-identical content across the pair, making their transplant close to a no-op. We use 405 both-correct pairs for the  $A_1$  channel and 436 for the  $A_2$  channel, sampled as in the 1-fact protocol (Section 4).

**Transplant scopes and metrics.** For each pair we transplant the donor’s KV cache, at the chosen positions, into the target’s forward pass, at three scopes: (i) *whole*, all 53 filler positions; (ii) the addend’s decoded positions ( $\theta = 0.15$ ); and (iii) the complement (all filler positions not labeled for that addend). We report two quantities, matching the main-text transplants: the *rank shift*, the improvement in the rank of the donor answer  $A_1^d + A_2$  in the target’s next-token distribution relative to its unmodified baseline rank (positive = the donor answer became more likely); and the *full-swap rate*, the fraction of targets whose top output token becomes the donor’s answer outright.

**Results.** Table 1 reports all three scopes for both channels. The causal effect concentrates where the heatmap places the value. Transplanting only the decoded positions recovers 93% ( $A_1$ ) and 85% ( $A_2$ ) of the corresponding whole-filler rank shift, despite using only 12 and 16 of the 53 positions. The complementary positions—the majority of the filler region—move the rank by only 12 places and essentially never produce a full swap (0.0% for  $A_1$ , 0.2% for  $A_2$ ). (The decoded and complement contributions need not sum to the whole-filler shift, since the transplant is nonlinear in the number of positions perturbed.) Consistent with the system-of-equations result, two-operand transplants perturb the target’s computation more often than they cleanly replace it, resulting in few full-swaps. The  $A_1$  channel moves the answer somewhat more readily than  $A_2$  at every scope (whole-filler swap 7.4% vs. 2.8%), in line with  $A_1$  being the more strongly encoded addend. Each addend is independently and positionally manipulable, and the positions the lens reads as carrying neither addend carry essentially no causal load, so the positional layout is a causal feature of the computation rather than an artifact of the logit lens.

Table 1: Position-resolved 2-fact transplant (DeepSeek V3,  $k = 50$  dots). *Rank shift*: improvement in the donor answer’s rank in the target’s next-token distribution relative to the unmodified baseline rank (given per channel); higher is a stronger effect. *Shift recovered*: rank shift as a fraction of the same channel’s whole-filler shift. *Full-swap*: fraction of targets that output the donor’s answer outright. 95% CIs in brackets. The  $A_1$  channel uses 405 fact-matched pairs; the  $A_2$  channel uses 436.

| Channel                 | Scope                            | # pos. | Rank shift  | Full-swap (%)   |
|-------------------------|----------------------------------|--------|-------------|-----------------|
| $A_1$<br>(base rank 88) | all filler                       | 53     | 70 [61, 85] | 7.4 [4.9, 10.1] |
|                         | $A_1$ -decoded ( $\theta=0.15$ ) | 12     | 65 [54, 76] | 4.4 [2.5, 6.7]  |
|                         | complement                       | 41     | 12 [9, 16]  | 0.0 [0.0, 0.0]  |
| $A_2$<br>(base rank 90) | all filler                       | 53     | 58 [44, 73] | 2.8 [1.4, 4.4]  |
|                         | $A_2$ -decoded ( $\theta=0.15$ ) | 16     | 49 [38, 63] | 1.1 [0.2, 2.3]  |
|                         | complement                       | 37     | 12 [9, 16]  | 0.2 [0.0, 0.7]  |

## F LLM judge prompts

### Neutral judge prompt

For 1-fact addition (asks for 1 number) and 2-fact addition (asks for 2 numbers):

```
### system

These are potentially important tokens that were taken from a model's
internal state during a task. The tokens in this list are hints about what
specific numbers the model was 'thinking about' in its internal state while
doing the calculation, when it has not yet gotten to outputting the final
answer. The original prompt contained formatting words like 'Filler' and
'Answer' that are not relevant to the task. The model was trained on Chinese
data, so it may use Chinese tokens as well as English.

What specific two numbers was the model thinking about?

Think briefly, then end your response with a JSON object on its own line:
{"n1": <int>, "confidence_n1": <float 0-1>, "n2": <int>, "confidence_n2":
<float 0-1>, "backups": [<int>, <int>, ... up to 10 ranked alternatives]}.
If you cannot determine n1 or n2, output null for that field.

### user

Top tokens (rank. score 'string'):
1. 37.275 '硼'
2. 13.463 'Neptune'
3. 8.990 'filler'
4. 8.735 '93'
5. 7.005 '五号'
6. 6.122 '5'
7. 4.818 'Np'
8. 4.384 'Californ'
9. 3.987 'uranium'
10. 3.665 '.'
[... 40 more tokens]
```

For 2-hop letter position:

```
### system

These are potentially important tokens that were taken from a model's
internal state during a task. The tokens in this list are hints about what
specific concept, entity, or name the model was 'thinking about' in its
internal state while doing the calculation, when it has not yet gotten to
outputting the final answer. The original prompt contained formatting words
like 'Filler' and 'Answer' that are not relevant to the task. The model was
trained on Chinese data, so it may use Chinese tokens as well as English.

What specific thing (person, place, object, concept, etc.) was the model
thinking about?

Think briefly, then end your response with a JSON object on its own line:
{"answer": <string>, "confidence": <float 0-1>, "backups": [<string>,
<string>, ... up to 10 ranked alternatives]}. If you cannot determine an
answer, output null for that field.

### user

[... 50 top tokens]
```

For the prompt sensitivity tests, the task-specific prompt is edited to name the expected target type: "What city was the model thinking about?" or "What chemical element was the model thinking about?"

**Prompt sensitivity.** We compare how different judge prompts affect results. A neutral prompt asks generically what the model was reasoning about; a task-specific prompt names the expected target type ("the name of a chemical element" or "the name of a city"). The task-specific prompt helps both judges where there is room to improve. On element names, Sonnet is already near ceiling under the neutral prompt and shows little change ( $\sim 0\%$ ), while Haiku gains 3–6%. On city names, a harder subtask where neither judge is saturated, the city-aware prompt helps both, with Sonnet gaining up to 11% and Haiku gaining up to 13%. This suggests the unsupervised pipeline can be made more accurate when an auditor has a hypothesis about what kind of information to search for, but does not require one for simple, clearly-defined intermediates. Full per-condition results under both prompts are in Tables 4 and 5.

## G Top- $K$ token accuracy

For numeric intermediates (1-fact and 2-fact addition), we restrict to numeric tokens and check exact rank. For string-valued intermediates (element and city names in the letter-position task), we use a top- $K$  substring coverage measure: each example has a multilingual reference dictionary containing the canonical English name, the chemical symbol where applicable, and a list of language aliases (English plus Chinese, since both models tokenize Chinese densely). A token at rank  $r$  is counted as a match if (i) it equals the chemical symbol exactly (case-sensitive, to avoid over-matching short English words), or (ii) it has a bidirectional case-insensitive substring overlap with any alias, with a minimum-length threshold of 4 characters for Latin-script tokens and 2 for CJK tokens. The bidirectional check captures both the case where the alias is a substring of the token and the case where the tokenizer splits the alias across multiple sub-tokens. Top- $K$  accuracy is the fraction of examples for which any match fires at rank  $\leq K$ . This baseline serves as a test of how much signal is in the aggregated tokens themselves versus how much is added by the judge.

## H Full decoding results (correct examples)

Tables 2–5 report the full top- $K$  decoding results used to evaluate how much task-relevant information is recoverable from the model residual stream after filler-token generation. For each experimental group, we report the sample size  $n$  and the percentage of examples for which the correct target appears within the top- $K$  decoded candidates, for  $K \in \{1, 2, 3, 5, 10\}$ .

We include three decoding procedures. In the direct match setting, we aggregate residual-stream token scores and check whether the target appears among the top- $K$  decoded tokens. For addition tasks, this is computed over the top- $K$  numeric tokens. For letter-position tasks, this is computed over the top- $K$  string tokens, allowing matches in either English or Chinese. In the *Haiku* and *Sonnet* judge settings, we instead ask a Claude model to infer the most likely latent target from the decoded candidates; top- $K$  accuracy is computed by checking whether the target appears in the judge’s primary guess or first  $K-1$  backup guesses. The neutral judge prompt is task-agnostic, while the leading judge prompts provide the relevant domain framing, such as chemistry for element names or geography for capitals.

For the 1-fact addition task, the target is the single retrieved value  $A$ . For the 2-fact addition task, a trial is counted as correct only when both addends,  $A_1$  and  $A_2$ , are recovered within the top- $K$  set. For the letter-position task, the target is the latent entity whose name supplies the requested letter. These appendix tables provide the full breakdown by filler type, filler length, model, and judge prompt, complementing the summary results in the main text.

Tables 6–7 provide shuffled-token baseline controls. In these controls, each example’s decoded top-50 token list is replaced with the top-50 list from a different example drawn from the same model, task, and filler condition, while the judge is still evaluated against the original example’s ground truth. If the judge were succeeding mainly by exploiting a task-level prior or the prompt framing, accuracy would remain high under this shuffle. The near-zero shuffled accuracy therefore shows that judge

performance depends on information specific to the original example, not merely on generic task structure or prompt-induced guessing.

Table 2: Decoding accuracy on **1-fact addition** (DeepSeek V3, neutral judge prompt). Each cell is the top- $K$  hit rate as a percentage of  $n$ . Direct match: target value  $A$  in top- $K$  numeric tokens of the aggregated residuals. Haiku/Sonnet: judge’s primary guess plus first  $K-1$  backups contains the target.

| Group         | $n$   | Direct match top- $K$ (%) |      |      |      |      | Haiku judge top- $K$ (%) |      |      |      |      | Sonnet judge top- $K$ (%) |      |      |      |       |
|---------------|-------|---------------------------|------|------|------|------|--------------------------|------|------|------|------|---------------------------|------|------|------|-------|
|               |       | 1                         | 2    | 3    | 5    | 10   | 1                        | 2    | 3    | 5    | 10   | 1                         | 2    | 3    | 5    | 10    |
| dots_10       | 350   | 66.0                      | 80.6 | 86.9 | 92.9 | 95.7 | 74.6                     | 87.7 | 90.6 | 93.4 | 96.6 | 84.3                      | 92.9 | 95.1 | 98.3 | 98.9  |
| dots_25       | 359   | 65.5                      | 84.7 | 90.8 | 94.2 | 96.1 | 78.3                     | 91.9 | 93.6 | 95.3 | 97.5 | 88.3                      | 95.3 | 96.7 | 99.2 | 100.0 |
| dots_50       | 357   | 56.9                      | 81.2 | 88.0 | 94.1 | 95.8 | 66.1                     | 85.2 | 90.8 | 94.4 | 96.9 | 79.0                      | 91.9 | 95.8 | 97.5 | 98.9  |
| dots_100      | 367   | 40.6                      | 76.3 | 90.5 | 94.8 | 96.5 | 60.2                     | 86.4 | 92.9 | 96.2 | 98.1 | 74.4                      | 91.6 | 95.1 | 97.8 | 99.2  |
| counting_5    | 361   | 71.2                      | 86.4 | 91.7 | 96.7 | 98.3 | 79.8                     | 89.8 | 92.5 | 94.5 | 97.8 | 85.6                      | 95.8 | 97.5 | 98.6 | 99.4  |
| counting_25   | 368   | 71.5                      | 93.2 | 96.5 | 98.4 | 99.5 | 81.5                     | 93.8 | 96.5 | 97.8 | 98.6 | 88.9                      | 95.9 | 97.8 | 99.2 | 99.2  |
| counting_50   | 364   | 61.5                      | 91.2 | 96.4 | 98.1 | 98.9 | 74.7                     | 91.2 | 95.3 | 97.5 | 99.2 | 82.1                      | 95.3 | 97.8 | 98.6 | 99.2  |
| alphabet_10   | 351   | 68.9                      | 86.0 | 90.9 | 94.0 | 95.7 | 76.9                     | 89.2 | 91.7 | 94.3 | 96.9 | 86.0                      | 96.6 | 96.6 | 97.4 | 98.9  |
| alphabet_25   | 362   | 69.1                      | 83.7 | 90.1 | 93.1 | 94.5 | 76.0                     | 89.0 | 92.0 | 95.0 | 97.0 | 84.8                      | 93.1 | 96.1 | 98.1 | 99.2  |
| <b>Pooled</b> | 3,239 | 63.4                      | 84.8 | 91.3 | 95.2 | 96.8 | 74.2                     | 89.3 | 92.9 | 95.4 | 97.6 | 83.7                      | 94.3 | 96.5 | 98.3 | 99.2  |

Table 3: Decoding accuracy on **2-fact addition** (neutral judge prompt). Direct match: BOTH addends  $A_1$  and  $A_2$  in top- $K$  numeric tokens. Haiku/Sonnet:  $\{n_1, n_2, \text{backups}\}$  contains both  $A_1$  and  $A_2$  within top- $K$ .

|                    |       | Direct match top- $K$ (%) |      |      |      |      | Haiku judge top- $K$ (%) |      |      |      |      | Sonnet judge top- $K$ (%) |      |      |      |       |
|--------------------|-------|---------------------------|------|------|------|------|--------------------------|------|------|------|------|---------------------------|------|------|------|-------|
| Group              | $n$   | 1                         | 2    | 3    | 5    | 10   | 1                        | 2    | 3    | 5    | 10   | 1                         | 2    | 3    | 5    | 10    |
| <i>DeepSeek V3</i> |       |                           |      |      |      |      |                          |      |      |      |      |                           |      |      |      |       |
| dots_10            | 244   | 0.0                       | 34.0 | 44.7 | 55.7 | 63.1 | 0.0                      | 56.1 | 70.5 | 76.6 | 79.5 | 0.0                       | 82.4 | 89.8 | 91.8 | 93.0  |
| dots_25            | 245   | 0.0                       | 31.0 | 49.0 | 59.6 | 66.5 | 0.0                      | 53.1 | 65.7 | 75.1 | 82.9 | 0.0                       | 82.0 | 92.2 | 94.3 | 97.1  |
| dots_50            | 246   | 0.0                       | 29.3 | 44.7 | 60.2 | 71.5 | 0.0                      | 48.8 | 66.7 | 74.8 | 85.8 | 0.0                       | 86.6 | 94.3 | 96.3 | 97.2  |
| counting_10        | 235   | 0.0                       | 37.0 | 59.1 | 72.3 | 80.4 | 0.0                      | 43.4 | 60.4 | 70.2 | 79.1 | 0.0                       | 78.3 | 89.8 | 93.2 | 96.2  |
| counting_25        | 257   | 0.0                       | 45.1 | 61.5 | 74.7 | 89.5 | 0.0                      | 55.3 | 71.2 | 77.8 | 83.7 | 0.0                       | 82.1 | 89.9 | 92.6 | 95.3  |
| counting_50        | 233   | 0.0                       | 35.2 | 55.8 | 76.8 | 88.0 | 0.0                      | 49.4 | 70.0 | 79.8 | 82.8 | 0.0                       | 82.4 | 89.7 | 92.7 | 94.0  |
| <b>DSv3 pooled</b> | 1,460 | 0.0                       | 35.3 | 52.5 | 66.5 | 76.5 | 0.0                      | 51.1 | 67.5 | 75.8 | 82.3 | 0.0                       | 82.3 | 91.0 | 93.5 | 95.5  |
| <i>Kimi K2</i>     |       |                           |      |      |      |      |                          |      |      |      |      |                           |      |      |      |       |
| dots_10            | 324   | 0.0                       | 64.8 | 76.5 | 83.0 | 86.7 | 0.0                      | 71.0 | 83.6 | 88.6 | 93.5 | 0.0                       | 84.0 | 88.9 | 93.5 | 96.0  |
| dots_25            | 351   | 0.0                       | 71.8 | 82.3 | 85.2 | 88.3 | 0.0                      | 72.4 | 82.3 | 84.9 | 89.2 | 0.0                       | 86.3 | 92.0 | 94.0 | 96.6  |
| dots_50            | 336   | 0.0                       | 77.4 | 91.4 | 95.2 | 96.4 | 0.0                      | 79.8 | 91.4 | 94.6 | 97.3 | 0.0                       | 94.3 | 97.6 | 99.1 | 100.0 |
| counting_10        | 333   | 0.0                       | 73.6 | 86.5 | 91.0 | 95.8 | 0.0                      | 78.4 | 88.9 | 93.4 | 97.6 | 0.0                       | 90.4 | 94.6 | 96.7 | 97.9  |
| counting_25        | 353   | 0.0                       | 75.6 | 91.8 | 96.9 | 99.7 | 0.0                      | 77.9 | 89.2 | 93.2 | 97.5 | 0.0                       | 93.8 | 97.5 | 98.3 | 99.7  |
| counting_50        | 356   | 0.0                       | 78.7 | 89.9 | 96.3 | 99.7 | 0.0                      | 81.7 | 88.2 | 91.0 | 94.4 | 0.0                       | 93.5 | 96.3 | 98.6 | 99.2  |
| <b>KK2 pooled</b>  | 2,053 | 0.0                       | 73.7 | 86.5 | 91.4 | 94.5 | 0.0                      | 76.9 | 87.3 | 90.9 | 94.9 | 0.0                       | 90.5 | 94.5 | 96.7 | 98.2  |

Table 4: Decoding accuracy on **letter-position task** with the *neutral* (task-agnostic) judge prompt. Elements: atomic-number-to-element entity. Capitals: country/state-to-capital entity. Direct match: substring of the entity name (English or Chinese alias, or chemical symbol for elements) appears in any top- $K$  token.

| Group                   | $n$   | Direct match top- $K$ (%) |      |      |      |      | Haiku judge top- $K$ (%) |      |      |      |      | Sonnet judge top- $K$ (%) |      |       |       |       |
|-------------------------|-------|---------------------------|------|------|------|------|--------------------------|------|------|------|------|---------------------------|------|-------|-------|-------|
|                         |       | 1                         | 2    | 3    | 5    | 10   | 1                        | 2    | 3    | 5    | 10   | 1                         | 2    | 3     | 5     | 10    |
| <i>DSv3, Elements</i>   |       |                           |      |      |      |      |                          |      |      |      |      |                           |      |       |       |       |
| dots_10                 | 236   | 65.7                      | 77.1 | 83.5 | 88.6 | 93.2 | 81.4                     | 91.1 | 91.9 | 94.1 | 95.3 | 86.4                      | 96.2 | 97.9  | 99.6  | 100.0 |
| dots_25                 | 230   | 61.3                      | 72.2 | 79.6 | 88.7 | 94.3 | 87.0                     | 91.3 | 93.0 | 93.9 | 94.8 | 87.8                      | 98.3 | 99.1  | 100.0 | 100.0 |
| dots_50                 | 231   | 60.6                      | 71.9 | 79.7 | 85.7 | 93.1 | 87.9                     | 91.3 | 92.2 | 93.1 | 95.7 | 89.6                      | 97.0 | 99.1  | 100.0 | 100.0 |
| counting_10             | 237   | 72.6                      | 81.4 | 84.8 | 90.3 | 94.5 | 86.9                     | 90.3 | 92.4 | 93.7 | 95.4 | 88.6                      | 98.3 | 99.2  | 99.6  | 100.0 |
| counting_25             | 233   | 66.5                      | 77.7 | 83.3 | 88.4 | 93.6 | 85.8                     | 90.6 | 91.8 | 94.0 | 95.7 | 89.3                      | 97.9 | 98.7  | 99.6  | 99.6  |
| counting_50             | 236   | 64.8                      | 75.0 | 83.9 | 89.0 | 93.2 | 84.3                     | 90.3 | 91.1 | 92.4 | 93.2 | 87.7                      | 97.5 | 98.3  | 99.2  | 99.2  |
| <b>DSv3, El pooled</b>  | 1,403 | 65.3                      | 75.9 | 82.5 | 88.5 | 93.7 | 85.5                     | 90.8 | 92.1 | 93.5 | 95.0 | 88.2                      | 97.5 | 98.7  | 99.6  | 99.8  |
| <i>DSv3, Capitals</i>   |       |                           |      |      |      |      |                          |      |      |      |      |                           |      |       |       |       |
| dots_10                 | 218   | 45.4                      | 58.3 | 61.9 | 70.6 | 78.4 | 64.2                     | 78.9 | 82.1 | 84.4 | 84.9 | 69.3                      | 86.2 | 88.1  | 90.4  | 90.8  |
| dots_25                 | 216   | 44.9                      | 57.9 | 61.6 | 67.6 | 76.9 | 69.4                     | 81.5 | 82.9 | 85.2 | 85.6 | 70.8                      | 88.0 | 89.4  | 90.7  | 91.2  |
| dots_50                 | 216   | 46.8                      | 53.2 | 62.5 | 69.4 | 76.4 | 67.6                     | 77.8 | 81.5 | 84.7 | 85.2 | 73.6                      | 86.1 | 88.4  | 90.7  | 92.1  |
| counting_10             | 214   | 47.2                      | 55.6 | 60.7 | 68.2 | 75.7 | 73.8                     | 84.1 | 86.9 | 90.2 | 90.7 | 79.0                      | 87.4 | 87.4  | 90.7  | 90.7  |
| counting_25             | 218   | 46.3                      | 58.7 | 61.9 | 69.7 | 76.6 | 69.7                     | 79.8 | 82.1 | 84.9 | 86.2 | 76.1                      | 87.2 | 88.1  | 90.4  | 90.8  |
| counting_50             | 217   | 47.0                      | 62.2 | 65.4 | 69.1 | 76.0 | 71.4                     | 81.1 | 83.4 | 85.3 | 86.2 | 77.9                      | 90.8 | 91.2  | 92.6  | 94.0  |
| <b>DSv3, Cap pooled</b> | 1,299 | 46.3                      | 57.7 | 62.4 | 69.1 | 76.7 | 69.4                     | 80.5 | 83.1 | 85.8 | 86.5 | 74.4                      | 87.6 | 88.8  | 90.9  | 91.6  |
| <i>KK2, Elements</i>    |       |                           |      |      |      |      |                          |      |      |      |      |                           |      |       |       |       |
| dots_10                 | 237   | 57.8                      | 73.8 | 81.0 | 89.0 | 96.2 | 89.9                     | 92.4 | 93.7 | 95.4 | 96.2 | 88.6                      | 95.8 | 98.7  | 98.7  | 98.7  |
| dots_25                 | 229   | 59.4                      | 72.9 | 79.9 | 90.0 | 94.8 | 90.0                     | 92.6 | 92.6 | 95.2 | 96.1 | 92.6                      | 97.4 | 98.3  | 99.1  | 99.1  |
| dots_50                 | 247   | 55.1                      | 68.4 | 74.1 | 84.2 | 91.9 | 87.4                     | 91.5 | 92.7 | 93.5 | 95.1 | 93.5                      | 98.4 | 99.2  | 99.6  | 99.6  |
| counting_10             | 226   | 62.4                      | 76.1 | 83.2 | 89.8 | 94.7 | 90.7                     | 92.9 | 94.7 | 96.9 | 97.3 | 92.5                      | 98.7 | 99.6  | 99.6  | 99.6  |
| counting_25             | 249   | 55.4                      | 73.9 | 81.1 | 87.6 | 93.2 | 88.0                     | 92.0 | 93.6 | 94.8 | 96.8 | 91.6                      | 97.2 | 98.8  | 99.6  | 99.6  |
| counting_50             | 240   | 59.6                      | 73.3 | 80.8 | 86.2 | 92.9 | 87.5                     | 91.7 | 92.9 | 95.8 | 96.7 | 92.5                      | 99.2 | 100.0 | 100.0 | 100.0 |
| <b>KK2, El pooled</b>   | 1,428 | 58.2                      | 73.0 | 80.0 | 87.7 | 93.9 | 88.9                     | 92.2 | 93.3 | 95.2 | 96.4 | 91.9                      | 97.8 | 99.1  | 99.4  | 99.4  |
| <i>KK2, Capitals</i>    |       |                           |      |      |      |      |                          |      |      |      |      |                           |      |       |       |       |
| dots_10                 | 260   | 49.2                      | 58.5 | 62.7 | 67.7 | 73.5 | 64.6                     | 80.4 | 83.8 | 87.3 | 87.3 | 68.5                      | 86.5 | 86.5  | 88.5  | 88.5  |
| dots_25                 | 260   | 47.7                      | 58.1 | 62.7 | 67.7 | 73.5 | 64.2                     | 77.7 | 82.7 | 85.4 | 85.8 | 66.2                      | 81.5 | 83.8  | 86.2  | 86.2  |
| dots_50                 | 261   | 47.1                      | 57.9 | 62.5 | 68.6 | 73.9 | 71.3                     | 81.6 | 83.1 | 84.7 | 85.8 | 69.0                      | 86.6 | 88.1  | 91.6  | 92.0  |
| counting_10             | 261   | 49.4                      | 60.5 | 65.9 | 69.0 | 74.3 | 69.0                     | 83.1 | 86.2 | 87.4 | 87.4 | 72.4                      | 86.6 | 87.7  | 90.0  | 90.0  |
| counting_25             | 255   | 49.0                      | 62.7 | 66.7 | 69.8 | 74.1 | 71.4                     | 81.6 | 84.7 | 87.1 | 87.8 | 74.1                      | 88.2 | 89.0  | 90.6  | 90.6  |
| counting_50             | 266   | 48.1                      | 61.3 | 65.4 | 66.5 | 70.7 | 71.4                     | 83.5 | 85.0 | 85.3 | 85.7 | 71.1                      | 84.6 | 86.5  | 87.2  | 87.6  |
| <b>KK2, Cap pooled</b>  | 1,563 | 48.4                      | 59.8 | 64.3 | 68.2 | 73.3 | 68.7                     | 81.3 | 84.3 | 86.2 | 86.6 | 70.2                      | 85.7 | 86.9  | 89.0  | 89.1  |

Table 5: Decoding accuracy on **letter-position task with the leading judge prompts** (*chemistry* for elements, *geography* for capitals). Direct-match column is identical to Table 4 since the residual tokens are the same; only the judge framing differs.

| Group                   | <i>n</i> | Direct match top- <i>K</i> (%) |      |      |      |      | Haiku judge top- <i>K</i> (%) |      |      |      |      | Sonnet judge top- <i>K</i> (%) |      |      |      |      |
|-------------------------|----------|--------------------------------|------|------|------|------|-------------------------------|------|------|------|------|--------------------------------|------|------|------|------|
|                         |          | 1                              | 2    | 3    | 5    | 10   | 1                             | 2    | 3    | 5    | 10   | 1                              | 2    | 3    | 5    | 10   |
| <i>DSv3, Elements</i>   |          |                                |      |      |      |      |                               |      |      |      |      |                                |      |      |      |      |
| dots_10                 | 236      | 65.7                           | 77.1 | 83.5 | 88.6 | 93.2 | 83.9                          | 95.3 | 98.3 | 98.7 | 98.7 | 85.6                           | 96.2 | 97.9 | 98.7 | 98.7 |
| dots_25                 | 230      | 61.3                           | 72.2 | 79.6 | 88.7 | 94.3 | 87.4                          | 96.5 | 97.4 | 98.3 | 98.3 | 90.0                           | 98.3 | 99.1 | 99.6 | 99.6 |
| dots_50                 | 231      | 60.6                           | 71.9 | 79.7 | 85.7 | 93.1 | 88.3                          | 95.2 | 97.8 | 98.3 | 98.3 | 89.6                           | 97.4 | 98.7 | 99.1 | 99.1 |
| counting_10             | 237      | 72.6                           | 81.4 | 84.8 | 90.3 | 94.5 | 87.8                          | 95.4 | 98.3 | 98.7 | 98.7 | 89.9                           | 97.9 | 97.9 | 98.3 | 98.3 |
| counting_25             | 233      | 66.5                           | 77.7 | 83.3 | 88.4 | 93.6 | 87.1                          | 96.6 | 98.3 | 99.1 | 99.1 | 89.7                           | 97.4 | 98.3 | 98.7 | 98.7 |
| counting_50             | 236      | 64.8                           | 75.0 | 83.9 | 89.0 | 93.2 | 85.2                          | 94.9 | 97.0 | 97.9 | 97.9 | 89.4                           | 97.9 | 98.7 | 98.7 | 98.7 |
| <b>DSv3, El pooled</b>  | 1,403    | 65.3                           | 75.9 | 82.5 | 88.5 | 93.7 | 86.6                          | 95.7 | 97.9 | 98.5 | 98.5 | 89.0                           | 97.5 | 98.4 | 98.9 | 98.9 |
| <i>DSv3, Capitals</i>   |          |                                |      |      |      |      |                               |      |      |      |      |                                |      |      |      |      |
| dots_10                 | 218      | 45.4                           | 58.3 | 61.9 | 70.6 | 78.4 | 83.5                          | 89.4 | 90.4 | 91.7 | 91.7 | 87.6                           | 92.7 | 93.1 | 94.0 | 95.0 |
| dots_25                 | 216      | 44.9                           | 57.9 | 61.6 | 67.6 | 76.9 | 84.3                          | 88.9 | 90.7 | 91.2 | 91.2 | 88.0                           | 94.0 | 94.9 | 95.8 | 95.8 |
| dots_50                 | 216      | 46.8                           | 53.2 | 62.5 | 69.4 | 76.4 | 85.2                          | 89.4 | 90.7 | 91.7 | 91.7 | 88.9                           | 91.7 | 92.6 | 93.5 | 94.4 |
| counting_10             | 214      | 47.2                           | 55.6 | 60.7 | 68.2 | 75.7 | 86.0                          | 91.1 | 92.1 | 93.0 | 93.0 | 89.7                           | 92.5 | 93.9 | 94.4 | 94.4 |
| counting_25             | 218      | 46.3                           | 58.7 | 61.9 | 69.7 | 76.6 | 84.9                          | 89.0 | 90.4 | 90.8 | 90.8 | 86.7                           | 92.2 | 93.1 | 94.0 | 94.0 |
| counting_50             | 217      | 47.0                           | 62.2 | 65.4 | 69.1 | 76.0 | 86.2                          | 89.9 | 91.2 | 92.2 | 92.2 | 90.3                           | 94.5 | 95.9 | 96.3 | 96.3 |
| <b>DSv3, Cap pooled</b> | 1,299    | 46.3                           | 57.7 | 62.4 | 69.1 | 76.7 | 85.0                          | 89.6 | 90.9 | 91.8 | 91.8 | 88.5                           | 92.9 | 93.9 | 94.7 | 95.0 |
| <i>KK2, Elements</i>    |          |                                |      |      |      |      |                               |      |      |      |      |                                |      |      |      |      |
| dots_10                 | 237      | 57.8                           | 73.8 | 81.0 | 89.0 | 96.2 | 90.7                          | 96.6 | 98.3 | 98.7 | 98.7 | 89.9                           | 95.4 | 97.5 | 97.9 | 97.9 |
| dots_25                 | 229      | 59.4                           | 72.9 | 79.9 | 90.0 | 94.8 | 90.4                          | 95.6 | 98.3 | 98.7 | 98.7 | 90.4                           | 95.2 | 98.3 | 98.7 | 98.7 |
| dots_50                 | 247      | 55.1                           | 68.4 | 74.1 | 84.2 | 91.9 | 88.7                          | 94.3 | 96.8 | 98.0 | 98.0 | 89.1                           | 95.1 | 98.0 | 98.8 | 98.8 |
| counting_10             | 226      | 62.4                           | 76.1 | 83.2 | 89.8 | 94.7 | 92.0                          | 96.9 | 99.1 | 99.1 | 99.1 | 91.6                           | 97.8 | 99.1 | 99.1 | 99.1 |
| counting_25             | 249      | 55.4                           | 73.9 | 81.1 | 87.6 | 93.2 | 89.6                          | 94.4 | 97.6 | 98.8 | 98.8 | 90.4                           | 96.8 | 98.8 | 98.8 | 98.8 |
| counting_50             | 240      | 59.6                           | 73.3 | 80.8 | 86.2 | 92.9 | 89.6                          | 95.8 | 97.9 | 98.8 | 98.8 | 90.4                           | 96.7 | 98.8 | 98.8 | 98.8 |
| <b>KK2, El pooled</b>   | 1,428    | 58.2                           | 73.0 | 80.0 | 87.7 | 93.9 | 90.1                          | 95.6 | 98.0 | 98.7 | 98.7 | 90.3                           | 96.1 | 98.4 | 98.7 | 98.7 |
| <i>KK2, Capitals</i>    |          |                                |      |      |      |      |                               |      |      |      |      |                                |      |      |      |      |
| dots_10                 | 260      | 49.2                           | 58.5 | 62.7 | 67.7 | 73.5 | 80.0                          | 87.3 | 88.8 | 89.2 | 89.2 | 85.4                           | 90.8 | 93.1 | 94.6 | 95.0 |
| dots_25                 | 260      | 47.7                           | 58.1 | 62.7 | 67.7 | 73.5 | 81.2                          | 88.8 | 89.2 | 89.2 | 89.2 | 86.5                           | 91.2 | 91.5 | 91.9 | 93.1 |
| dots_50                 | 261      | 47.1                           | 57.9 | 62.5 | 68.6 | 73.9 | 82.4                          | 87.7 | 88.1 | 88.1 | 88.1 | 87.4                           | 92.0 | 93.9 | 95.0 | 95.4 |
| counting_10             | 261      | 49.4                           | 60.5 | 65.9 | 69.0 | 74.3 | 82.0                          | 87.0 | 88.5 | 89.7 | 89.7 | 85.8                           | 91.2 | 93.1 | 93.9 | 94.3 |
| counting_25             | 255      | 49.0                           | 62.7 | 66.7 | 69.8 | 74.1 | 85.5                          | 89.8 | 90.2 | 90.2 | 90.2 | 86.7                           | 92.2 | 92.5 | 93.7 | 94.1 |
| counting_50             | 266      | 48.1                           | 61.3 | 65.4 | 66.5 | 70.7 | 80.1                          | 87.6 | 88.3 | 88.3 | 88.3 | 86.5                           | 91.4 | 92.5 | 93.2 | 93.2 |
| <b>KK2, Cap pooled</b>  | 1,563    | 48.4                           | 59.8 | 64.3 | 68.2 | 73.3 | 81.8                          | 88.0 | 88.9 | 89.1 | 89.1 | 86.4                           | 91.4 | 92.8 | 93.7 | 94.2 |

Table 6: Shuffled-token control on DeepSeek V3. For each example, the top-50 token list is replaced with another example’s top-50 list from the same task and filler condition; the judge is scored against the original example’s ground truth. Rows compare the original residual-decode accuracy, the shuffled-control accuracy, and the resulting change  $\Delta$ , on dots\_10.

| Task         | Judge  | Prompt  | Method   | $n$ | 1     | 2     | 3     | 5     | 10    |
|--------------|--------|---------|----------|-----|-------|-------|-------|-------|-------|
| 1-fact       | Haiku  | neutral | orig.    | 350 | 74.6  | 87.7  | 90.6  | 93.4  | 96.6  |
|              |        |         | shuf.    | 350 | 0.6   | 1.4   | 2.0   | 3.1   | 4.3   |
|              |        |         | $\Delta$ |     | -74.0 | -86.3 | -88.6 | -90.3 | -92.3 |
| 1-fact       | Sonnet | neutral | orig.    | 350 | 84.3  | 92.9  | 95.1  | 98.3  | 98.9  |
|              |        |         | shuf.    | 350 | 0.6   | 1.1   | 2.0   | 2.9   | 4.9   |
|              |        |         | $\Delta$ |     | -83.7 | -91.7 | -93.1 | -95.4 | -94.0 |
| 2-fact       | Haiku  | neutral | orig.    | 244 | –     | 56.1  | 70.5  | 76.6  | 79.5  |
|              |        |         | shuf.    | 244 | –     | 0.0   | 0.0   | 0.0   | 0.0   |
|              |        |         | $\Delta$ |     | –     | -56.1 | -70.5 | -76.6 | -79.5 |
| 2-fact       | Sonnet | neutral | orig.    | 244 | –     | 82.4  | 89.8  | 91.8  | 93.0  |
|              |        |         | shuf.    | 244 | –     | 0.0   | 0.0   | 0.0   | 0.4   |
|              |        |         | $\Delta$ |     | –     | -82.4 | -89.8 | -91.8 | -92.6 |
| Letter-pos.  | Haiku  | neutral | orig.    | 236 | 81.4  | 91.1  | 91.9  | 94.1  | 95.3  |
|              |        |         | shuf.    | 236 | 0.4   | 0.8   | 0.8   | 1.3   | 1.3   |
|              |        |         | $\Delta$ |     | -80.9 | -90.3 | -91.1 | -92.8 | -94.1 |
| Letter-pos.  | Sonnet | neutral | orig.    | 236 | 86.4  | 96.2  | 97.9  | 99.6  | 100.0 |
|              |        |         | shuf.    | 236 | 0.4   | 1.3   | 1.3   | 1.3   | 1.3   |
|              |        |         | $\Delta$ |     | -86.0 | -94.9 | -96.6 | -98.3 | -98.7 |
| Letter-pos.  | Haiku  | chem.   | orig.    | 236 | 83.9  | 95.3  | 98.3  | 98.7  | 98.7  |
|              |        |         | shuf.    | 236 | 0.4   | 0.8   | 2.1   | 2.5   | 2.5   |
|              |        |         | $\Delta$ |     | -83.5 | -94.5 | -96.2 | -96.2 | -96.2 |
| Letter-pos.  | Sonnet | chem.   | orig.    | 236 | 85.6  | 96.2  | 97.9  | 98.7  | 98.7  |
|              |        |         | shuf.    | 236 | 0.4   | 1.3   | 2.5   | 2.5   | 2.5   |
|              |        |         | $\Delta$ |     | -85.2 | -94.9 | -95.3 | -96.2 | -96.2 |
| Capital-pos. | Haiku  | neutral | orig.    | 218 | 64.2  | 78.9  | 82.1  | 84.4  | 84.9  |
|              |        |         | shuf.    | 218 | 0.5   | 0.5   | 0.5   | 0.5   | 0.5   |
|              |        |         | $\Delta$ |     | -63.8 | -78.4 | -81.7 | -83.9 | -84.4 |
| Capital-pos. | Sonnet | neutral | orig.    | 218 | 69.3  | 86.2  | 88.1  | 90.4  | 90.8  |
|              |        |         | shuf.    | 218 | 0.5   | 0.5   | 0.5   | 0.5   | 0.5   |
|              |        |         | $\Delta$ |     | -68.8 | -85.8 | -87.6 | -89.9 | -90.4 |
| Capital-pos. | Haiku  | geo.    | orig.    | 218 | 83.5  | 89.4  | 90.4  | 91.7  | 91.7  |
|              |        |         | shuf.    | 218 | 0.0   | 0.0   | 0.5   | 0.5   | 0.5   |
|              |        |         | $\Delta$ |     | -83.5 | -89.4 | -89.9 | -91.3 | -91.3 |
| Capital-pos. | Sonnet | geo.    | orig.    | 218 | 87.6  | 92.7  | 93.1  | 94.0  | 95.0  |
|              |        |         | shuf.    | 218 | 0.0   | 0.0   | 0.5   | 0.9   | 0.9   |
|              |        |         | $\Delta$ |     | -87.6 | -92.7 | -92.7 | -93.1 | -94.0 |

Table 7: Shuffled-token control on Kimi K2. The setup is the same as in Table 6: shuffled rows replace each example’s decoded top-50 tokens with another example’s top-50 list from the same task and filler condition, while keeping the original ground truth.

| Task         | Judge  | Prompt  | Method   | $n$ | 1     | 2     | 3     | 5     | 10    |
|--------------|--------|---------|----------|-----|-------|-------|-------|-------|-------|
| 2-fact       | Haiku  | neutral | orig.    | 330 | –     | 73.6  | 84.5  | 88.8  | 92.4  |
|              |        |         | shuf.    | 330 | –     | 0.3   | 0.3   | 0.3   | 0.6   |
|              |        |         | $\Delta$ |     | –     | -73.3 | -84.2 | -88.5 | -91.8 |
| 2-fact       | Sonnet | neutral | orig.    | 330 | –     | 86.7  | 93.9  | 96.7  | 97.6  |
|              |        |         | shuf.    | 330 | –     | 0.0   | 0.3   | 0.3   | 0.6   |
|              |        |         | $\Delta$ |     | –     | -86.7 | -93.6 | -96.4 | -97.0 |
| Letter-pos.  | Haiku  | neutral | orig.    | 237 | 86.1  | 88.6  | 91.6  | 91.6  | 92.8  |
|              |        |         | shuf.    | 237 | 0.8   | 0.8   | 0.8   | 1.3   | 1.7   |
|              |        |         | $\Delta$ |     | -85.2 | -87.8 | -90.7 | -90.3 | -91.1 |
| Letter-pos.  | Sonnet | neutral | orig.    | 237 | 88.6  | 95.8  | 97.5  | 99.6  | 100.0 |
|              |        |         | shuf.    | 237 | 1.3   | 1.3   | 2.5   | 3.8   | 3.8   |
|              |        |         | $\Delta$ |     | -87.3 | -94.5 | -94.9 | -95.8 | -96.2 |
| Letter-pos.  | Haiku  | chem.   | orig.    | 237 | 87.3  | 93.7  | 97.0  | 97.5  | 97.9  |
|              |        |         | shuf.    | 237 | 1.3   | 1.7   | 3.0   | 4.2   | 4.2   |
|              |        |         | $\Delta$ |     | -86.1 | -92.0 | -94.1 | -93.2 | -93.7 |
| Letter-pos.  | Sonnet | chem.   | orig.    | 237 | 88.6  | 96.6  | 99.2  | 99.2  | 99.6  |
|              |        |         | shuf.    | 237 | 1.3   | 3.4   | 3.8   | 5.5   | 5.5   |
|              |        |         | $\Delta$ |     | -87.3 | -93.2 | -95.4 | -93.7 | -94.1 |
| Capital-pos. | Haiku  | neutral | orig.    | 261 | 63.6  | 74.3  | 77.4  | 80.8  | 82.4  |
|              |        |         | shuf.    | 261 | 0.0   | 0.0   | 0.4   | 0.4   | 0.4   |
|              |        |         | $\Delta$ |     | -63.6 | -74.3 | -77.0 | -80.5 | -82.0 |
| Capital-pos. | Sonnet | neutral | orig.    | 261 | 62.1  | 78.9  | 82.8  | 85.4  | 85.8  |
|              |        |         | shuf.    | 261 | 0.4   | 0.4   | 0.4   | 0.4   | 0.4   |
|              |        |         | $\Delta$ |     | -61.7 | -78.5 | -82.4 | -85.1 | -85.4 |
| Capital-pos. | Haiku  | geo.    | orig.    | 261 | 77.0  | 87.7  | 89.3  | 89.3  | 89.3  |
|              |        |         | shuf.    | 261 | 0.4   | 0.8   | 0.8   | 0.8   | 0.8   |
|              |        |         | $\Delta$ |     | -76.6 | -87.0 | -88.5 | -88.5 | -88.5 |
| Capital-pos. | Sonnet | geo.    | orig.    | 261 | 84.3  | 90.0  | 91.2  | 92.3  | 92.7  |
|              |        |         | shuf.    | 261 | 0.4   | 0.8   | 0.8   | 1.1   | 1.5   |
|              |        |         | $\Delta$ |     | -83.9 | -89.3 | -90.4 | -91.2 | -91.2 |

## I System of equations: decoding results

This appendix gives the full decoding breakdown for the system-of-equations task on DeepSeek V3, pooled across six filler conditions (dot filler with  $k = 10, 25, 50$  and counting filler with  $k = 5, 10, 25$ ;  $n = 1,824$ ). Table 8 reports per-intermediate recovery at each top- $K$ ; Table 9 reports the shuffled-token control alongside the (near-identical) judge performance; Table 10 reports where the decode lands when the queried variable  $y$  is not the top-1 token.

The intermediate value  $y$  is recovered nearly always (top-5 94.8%), while the input  $x$ , although given in the prompt, is barely present in the filler residual (top-5 21.9%), and the two transient sub-products fall in between, with the later  $c_2y$  more strongly encoded than the earlier  $c_1x$ . At top-5 the ordering is  $y$  (94.8) >  $c_2y$  (85.6) > answer (65.0) >  $c_1x$  (41.8) >  $x$  (21.9). Because the prompt carries no informative context, the Haiku and Sonnet judges track direct token recovery to within  $\sim 1$  pp on every intermediate, in contrast to the retrieval tasks, where the judge adds up to tens of points.

Table 8: Per-intermediate decoding accuracy on the **system of equations** (DeepSeek V3, direct token recovery, pooled  $n = 1,824$  over six filler conditions). Each cell is the top- $K$  hit rate as a percentage of  $n$ .  $x$  is given in the prompt;  $c_1x$ ,  $y$ ,  $c_2y$ , and the answer are computed in context. The Haiku and Sonnet judges track these values to within  $\sim 1$  pp (Table 9).

| Target      | top-1 | top-2 | top-3 | top-5 | top-10 |
|-------------|-------|-------|-------|-------|--------|
| $c_1x$      | 3     | 11    | 22    | 42    | 64     |
| $y$         | 55    | 82    | 89    | 95    | 97     |
| $c_2y$      | 30    | 61    | 75    | 86    | 93     |
| $x$ (given) | 0     | 2     | 7     | 22    | 51     |
| answer      | 6     | 22    | 44    | 65    | 80     |

Table 9: Real vs. shuffled decoding on the **system of equations** (DeepSeek V3, top-5, pooled  $n = 1,824$ ). *Real*: direct / Haiku / Sonnet top-5 recovery. *Shuffled*: each example scored against another example’s decoded tokens (seed-0 derangement). The judges match direct recovery to within  $\sim 1$  pp because the prompt provides no context to exploit, and the shuffled control collapses to chance.

| Target | Real (direct / Haiku / Sonnet) | Shuffled (direct / Haiku / Sonnet) |
|--------|--------------------------------|------------------------------------|
| $c_1x$ | 41.8 / 42.9 / 42.3             | 1.7 / 2.0 / 2.0                    |
| $y$    | 94.8 / 95.3 / 95.6             | 1.3 / 1.7 / 1.6                    |
| $c_2y$ | 85.6 / 85.7 / 85.9             | 0.5 / 1.0 / 1.0                    |
| $x$    | 21.9 / 22.0 / 22.0             | 1.3 / 2.0 / 2.0                    |
| answer | 65.0 / 63.3 / 64.3             | 0.6 / 0.8 / 0.8                    |

Table 10: Where the decode lands when the middle variable  $y$  is not the top-1 token (DeepSeek V3, system of equations).  $y$  is top-1 in 55% of examples; for the remaining 812, the table gives the identity of the top-1 numeric token. The decode overwhelmingly lands on an adjacent step of the same chain.

| Top-1 identity (when not $y$ )   | Share |
|----------------------------------|-------|
| $c_2y$ (next value in the chain) | 67%   |
| answer                           | 14%   |
| other problem quantity           | 7%    |
| $c_1x$                           | 7%    |
| off-problem                      | 5%    |
| $x$ (base)                       | 0%    |

## J Full decoding results (incorrect examples)

Tables 11 and 12 extend the decoding analysis to incorrect examples on the dots-10 filler condition. The decoder runs identically on correct and incorrect examples; only the scoring requires correct examples to define the target. Direct top-2 token match, the most stringent measure of what surfaces in the residual stream, stays within  $\pm 10$  pp of correct-example accuracy on 5 of 7 (model, task) combinations, with the exceptions being 2-fact addition (where incorrect examples score higher) and capital-position (where they score lower). On 2-fact addition, both operands are in fact *more* recoverable on incorrect examples (DeepSeek V3: 34%  $\rightarrow$  55%; Kimi K2: 72%  $\rightarrow$  82% top-2 direct), because the sum  $A_1 + A_2$  is essentially absent from the residual stream when the model fails, surfacing in the top-2 numeric tokens 10–16 $\times$  more often on correct examples than incorrect ones (Table 12). With no sum competing for top- $K$  slots, the operands occupy them more readily. Both models exhibit the same asymmetry, suggesting “operands retrieved, sum not formed” is a property of the task structure rather than a quirk of either model. The exception is capital-position, where intermediate recovery does drop on incorrect examples (DeepSeek V3:  $-10$  pp direct; Kimi K2:  $-16$  pp), indicating that for some capital lookups the failure mode is missing retrieval rather than missing composition. Together, these results show that the decoder is diagnostic about *where* a computation fails. When retrieval succeeds but composition does not (2-fact), the operands surface clearly and the sum is missing; when retrieval itself fails (some capital-pos cases), the relevant intermediate is genuinely not in the residual stream to be decoded.

Table 11: Decoding accuracy on correct vs. incorrect examples (dots-10 filler, neutral judge prompt). *Direct*: target intermediate in top-2 decoded tokens (numeric for addition tasks, string-match for letter-position). *Haiku/Sonnet*: judge’s top-2 (primary guess plus first backup) contains the target. For 2-fact, both  $A_1$  and  $A_2$  must be present. The decoder runs identically on both subsets; only the scoring is restricted to examples with a well-defined target. Recovery rates on incorrect examples are within  $\pm 7$  pp of correct on most task–model combinations, with capital-pos as the main exception (where retrieval, not just composition, can fail).

| Task               | Subset    | $n$ | Direct (%) | Haiku (%) | Sonnet (%) |
|--------------------|-----------|-----|------------|-----------|------------|
| <i>DeepSeek V3</i> |           |     |            |           |            |
| 1-fact addition    | correct   | 350 | 80.6       | 87.7      | 92.9       |
| 1-fact addition    | incorrect | 150 | 72.0       | 78.7      | 86.0       |
| 2-fact addition    | correct   | 244 | 34.0       | 56.1      | 82.4       |
| 2-fact addition    | incorrect | 756 | 55.4       | 69.6      | 83.6       |
| Element-position   | correct   | 236 | 77.1       | 91.1      | 96.2       |
| Element-position   | incorrect | 49  | 83.7       | 95.9      | 95.9       |
| Capital-position   | correct   | 218 | 58.3       | 78.9      | 86.2       |
| Capital-position   | incorrect | 144 | 47.9       | 64.6      | 80.6       |
| <i>Kimi K2</i>     |           |     |            |           |            |
| 2-fact addition    | correct   | 330 | 72.4       | 73.6      | 86.7       |
| 2-fact addition    | incorrect | 670 | 81.8       | 83.4      | 88.8       |
| Element-position   | correct   | 237 | 67.9       | 88.6      | 95.8       |
| Element-position   | incorrect | 48  | 64.6       | 93.8      | 97.9       |
| Capital-position   | correct   | 261 | 57.5       | 74.3      | 78.9       |
| Capital-position   | incorrect | 101 | 41.6       | 53.5      | 67.3       |

Table 12: Recovery of the sum  $A_1 + A_2$  on 2-fact addition, correct vs. incorrect examples (dots-10 filler). The sum surfaces in the top- $K$  numeric tokens 10–16 $\times$  more often when the model answers correctly. On incorrect examples the sum is essentially absent from the residual stream, while  $A_1$  and  $A_2$  themselves remain recoverable (Table 11), identifying composition, not retrieval, as the 2-fact failure mode. The pattern holds for both models.

| Model       | Subset    | $n$ | Top-1 (%) | Top-2 (%) | Top-3 (%) | Top-5 (%) | Top-10 (%) |
|-------------|-----------|-----|-----------|-----------|-----------|-----------|------------|
| DeepSeek V3 | correct   | 244 | 5.3       | 21.7      | 34.4      | 44.3      | 54.1       |
| DeepSeek V3 | incorrect | 756 | 0.3       | 2.0       | 4.0       | 7.3       | 15.3       |
| Kimi K2     | correct   | 330 | 0.0       | 11.2      | 27.6      | 38.5      | 46.4       |
| Kimi K2     | incorrect | 670 | 0.0       | 0.7       | 4.0       | 9.3       | 14.0       |

## K Structure of decoding failures

Across all four tasks, the examples on which the decoder fails to surface the target among its top-2 tokens are not noise: the tokens that displace the target are almost always other quantities from the same computation. This appendix collects the failure analyses for each task, complementing the incorrect-example decoding in Appendix J.

**1-fact addition (DeepSeek V3).** The most common incorrect guess is the given addend  $X$  (the random two-digit number from the prompt), which appears as top-1 in 39–69% of these failure cases. The sum  $A+X$  appears in 16–29%. Direct neighbors ( $A \pm 1$ ,  $X \pm 1$ ,  $\text{sum} \pm 1$ ) account for an additional few percent. In other words, when the pipeline fails to surface  $A$ , it is almost always because the model is more strongly encoding  $X$  or the answer, not because it is outputting noise.

**2-fact addition (DeepSeek V3).** When neither  $A_1$  nor  $A_2$  is in the top-2, the most common numeric token is the sum  $A_1+A_2$  itself: for DeepSeek V3 dots-25, 41% of failures have the exact sum in top-2, and in 73% it is within  $\pm 5$ . Combined with  $A_1 \pm 1$  and  $A_2 \pm 1$  near-misses, structured failure modes account for  $\sim 60$ –80% of all failures. The model is partway through its computation and is representing the sum rather than the addends. This is not a pure failure as it is still related to the underlying computation. The same holds for the broader set of examples in which the top-2 numeric tokens miss at least one addend (DeepSeek V3, pooling all six filler conditions,  $n = 946$  of 1,460):  $A_1$  appears somewhere in the numeric list in 92.5% of these cases and  $A_2$  in 77.0%, and when  $A_2$  is present its median rank is 3, just outside the top-2 window (direct top-10 on this task is 76.5%). In 98.4% of these cases the top numeric tokens contain the sum or a value within  $\pm 1$  of an addend, against an 11% chance rate.  $A_2$  is usually the missing addend, which matches the logit-lens and transplant results.

**System of equations (DeepSeek V3).** As on the other tasks, “failures” are structured. When  $y$  is not the top-1 token, the decode most often surfaces the adjacent step of the same chain:  $c_2y$ , the sub-product one rung below the answer, in 67% of these cases, and the answer itself in a further 14%; only 5% is off-problem. The pipeline often returns every step of the multi-step computation. Within the top-10 tokens, every step ( $x$  through the answer) is found 51% of the time ( $x$ ) to 97% of the time ( $y$ ).

## L Residual-subtraction ablation

We ablate the residual-subtraction step by re-running the pipeline with raw logit-lens probabilities instead of residualized scores. Results on DeepSeek V3 2-fact addition show that residualization can help substantially: with dots-10 filler, Sonnet top-2 accuracy is 82.4% with residualization versus 73.8% without, an 8.6 pp gain. However, residualization is not uniformly beneficial. With counting filler, it hurts the Haiku judge by up to 11.5 pp, from 90.6% to 79.1% at top-10. This is because counting digits are themselves often relevant intermediates and the residual step suppresses them. Residualization helps when: (i) there is a strong cross-example baseline obscuring the signal (formatting, common artifacts); (ii) the decode target varies across examples; and (iii) the downstream judge does not need the suppressed tokens. Where any of these breaks, raw logit-lens readouts may be preferable. We report residualized results as the default but note the trade-off.

Table 13 ablates the residual-subtraction step used in the main decoding pipeline. In the residualized setting, token scores are computed by subtracting the cross-example mean score for each layer-position setting; in the no-residual ablation, tokens are ranked directly by their raw logit-lens probabilities. This comparison isolates when residualization helps remove common filler or formatting artifacts, and when it can instead suppress useful signal.

Residualization can be significantly beneficial, but not always. On DeepSeek V3 2-fact addition with dots\_10 filler, residualization improves Sonnet top-2 accuracy from 73.8% to 82.4%. However, with counting\_10 filler, residualization reduces Haiku top-10 accuracy from 90.6% to 79.1%. Inspecting the decoded tokens suggests this occurs because counting filler makes digit tokens part of the cross-example baseline, and those same numeric tokens can be relevant to the target intermediates.

Table 13: Effect of residualization on decoding accuracy. The residual fingerprint (Section 5) ranks tokens by  $P_e(\text{token} \mid s) - \text{mean}_e P_e(\text{token} \mid s)$  per layer–position setting  $s$ ; the ablation removes the cross-example mean subtraction and ranks by raw  $P_e(\text{token} \mid s)$  instead. Each cell is the LLM-judge top- $K$  hit rate (%). Row pairs compare the same task, condition, and judge, with the residual method on top and the ablation below. These results are for DeepSeek V3.

| Task        | Cond.       | Judge  | Method   | $n$ | 1    | 2    | 3    | 5    | 10   |
|-------------|-------------|--------|----------|-----|------|------|------|------|------|
| 2-fact      | dots_10     | Haiku  | residual | 244 | –    | 56.1 | 70.5 | 76.6 | 79.5 |
|             |             |        | no-resid | 244 | –    | 55.3 | 73.0 | 77.5 | 81.6 |
| 2-fact      | dots_10     | Sonnet | residual | 244 | –    | 82.4 | 89.8 | 91.8 | 93.0 |
|             |             |        | no-resid | 244 | –    | 73.8 | 85.2 | 88.1 | 90.2 |
| 2-fact      | counting_10 | Haiku  | residual | 235 | –    | 43.4 | 60.4 | 70.2 | 79.1 |
|             |             |        | no-resid | 235 | –    | 45.1 | 67.2 | 79.6 | 90.6 |
| 2-fact      | counting_10 | Sonnet | residual | 235 | –    | 78.3 | 89.8 | 93.2 | 96.2 |
|             |             |        | no-resid | 235 | –    | 68.1 | 83.0 | 88.9 | 94.0 |
| Letter-pos. | dots_10     | Haiku  | residual | 454 | 73.1 | 85.2 | 87.2 | 89.4 | 90.3 |
|             |             |        | no-resid | 454 | 75.1 | 82.0 | 84.6 | 86.6 | 88.5 |
| Letter-pos. | dots_10     | Sonnet | residual | 454 | 78.2 | 91.4 | 93.2 | 95.2 | 95.6 |
|             |             |        | no-resid | 454 | 83.9 | 92.0 | 93.0 | 95.2 | 96.1 |

## M LLM judge controls

The shuffled-token controls (Appendix H) show that judge accuracy depends on example-specific content in the decoded token list. The two controls here ask a finer question: given the correct example’s list, how much of the judge’s gain over direct token matching comes from information present in the list versus arithmetic or domain inference by the judge? Both controls use the Sonnet judge, the neutral 2-fact prompt (Appendix F), and the top-2 both-addends metric of Section 6, on 2-fact addition for DeepSeek V3 and Kimi K2.

**Numbers-only lists.** We strip every non-numeric token from each example’s aggregated top-50 list, leaving only the ranked numeric tokens with their scores. This removes chemical element names, name fragments, symbols, and any other contextual string tokens. Judge accuracy falls to the direct-token accuracy: with only a ranked list of numbers, the judge can do no better than read off the top of the list (Table 14). On DeepSeek V3, the worst case, Sonnet falls from 82.3% to 36.4%, against 35.2% for direct top-2; on Kimi K2 it falls from 90.0% to 73.1%, against 74.1% direct. Across the other conditions the numbers-only judge stays at roughly 60–85% and always tracks the corresponding direct-token accuracy.

**Addends removed.** We keep every string token and every numeric token other than  $A_1$  and  $A_2$ , and replace each occurrence of the two true addends with a random integer drawn from  $[1, 100]$ . The contextual tokens (element names and fragments) are untouched. Accuracy again drops sharply, to 39.2% on DeepSeek V3 and 28.2% on Kimi K2 (Table 14). The context tokens alone are therefore insufficient: the judge relies on the addends actually being present in the list.

Table 14: Judge controls on **2-fact addition** (Sonnet judge, neutral prompt, both addends in top-2, pooled across filler conditions). *Full list*: the standard pipeline output. *Numbers only*: all non-numeric tokens removed. *Addends removed*:  $A_1$  and  $A_2$  replaced with random integers in  $[1, 100]$ , all other tokens kept. *Direct*: no judge, both addends in the top-2 numeric tokens.

| Condition               | DeepSeek V3 (%) | Kimi K2 (%) |
|-------------------------|-----------------|-------------|
| Direct top-2 (no judge) | 35.2            | 74.1        |
| Sonnet, full list       | 82.3            | 90.0        |
| Sonnet, numbers only    | 36.4            | 73.1        |
| Sonnet, addends removed | 39.2            | 28.2        |

Together, the two controls show that the judge’s uplift over direct matching comes from context-providing tokens such as the names and name fragments of chemical elements, and that this context is only useful when the actual  $A_1$  and  $A_2$  numeric tokens are also present. The judge is choosing among candidates that the residual stream has surfaced, not generating them. This is consistent with the failure analysis in Section 6: among DeepSeek V3 examples in which the top-2 numeric tokens miss at least one addend ( $n = 946$  of 1,460 pooled over six filler conditions),  $A_1$  is present somewhere in the numeric list in 92.5% of cases and  $A_2$  in 77.0%, with a median rank of 3 for  $A_2$  when present, and 98.4% of these lists contain the sum or a value within  $\pm 1$  of an addend (chance: 11%). The absence of a judge does not affect any of the mechanistic results, and on the system-of-equations task, which offers no usable context, both judges match direct recovery (Appendix I).

## N Robustness of the logit-lens readout: precision and lens choice

All mechanistic and decoding results in the main text use 4-bit quantized checkpoints (Appendix B) and the plain logit lens. This appendix checks both choices. To check that the residual-stream geometry read by the logit lens is not an artifact of quantization, we first repeated the DeepSeek V3 2-fact addition analysis on the same 1,000 dots-10 examples using the official native-FP8 DeepSeek-V3-0324 checkpoint [35] instead of the 4-bit AWQ checkpoint.

**Logit lens.** The heatmap structure of Figure 3 is unchanged.  $A_1$ ,  $A_2$ , and  $A_1 + A_2$  are all recoverable from filler-position residuals in layers 30–60 under FP8, at comparable strength to AWQ. The peak positions for  $A_1$  and for the sum are identical across the two checkpoints; the  $A_2$  peak shifts four filler positions later under FP8.

**Decoding.** Table 15 compares direct and judge-assisted recovery on correct examples. Direct recovery of both addends among the top-10 numeric tokens is 63.1% under AWQ and 63.6% under FP8; under the stricter top-2 metric it is 34.0% and 28.6%. As in the main text, examples that fail the exact-both criterion still contain computation-relevant information: among top-2 failures, at least one exact addend appears in 93.2% (AWQ) and 95.2% (FP8) of cases, the sum appears in 32.9% and 36.4%, and 96.3% and 97.6% contain an exact addend, a  $\pm 1$  near miss, or the sum. Judge-assisted recovery is also stable: Sonnet recovers both addends at top-2 in 82.4% (AWQ) and 86.6% (FP8) of examples and at top-10 in 93.0% and 95.2%; Haiku shows the same stability.

Table 15: DeepSeek V3, **2-fact addition**, dots-10, correct examples: 4-bit AWQ checkpoint (used throughout the paper) versus the native-FP8 DeepSeek-V3-0324 checkpoint. *Direct*: both addends among the top- $K$  numeric tokens. *Sonnet*: judge’s top- $K$  contains both addends. Failure rows are computed over the examples that fail direct top-2.

| Metric                                  | AWQ (4-bit) (%) | FP8 (%) |
|-----------------------------------------|-----------------|---------|
| Direct top-2                            | 34.0            | 28.6    |
| Direct top-10                           | 63.1            | 63.6    |
| Sonnet top-2                            | 82.4            | 86.6    |
| Sonnet top-10                           | 93.0            | 95.2    |
| <i>Among direct top-2 failures</i>      |                 |         |
| $\geq 1$ exact addend in numeric list   | 93.2            | 95.2    |
| Sum $A_1 + A_2$ in numeric list         | 32.9            | 36.4    |
| Exact addend, $\pm 1$ near miss, or sum | 96.3            | 97.6    |

**Lens choice.** The logit lens reads the residual stream through the unembedding alone. A tuned lens [21] fits per-layer probes to reproduce the model’s final-layer distribution, which biases it toward the eventual output rather than intermediate values that are, by construction, not the output. The Jacobian lens (J-lens) of Gurnee et al. [26] instead reads a state through the average Jacobian of a late residual stream with respect to the layer being read, so it captures how a perturbation at that layer would propagate through the rest of the network. To check that the logit lens is not missing signal that a J-lens would surface, we fit a J-lens on DeepSeek V3 (4-bit AWQ; 100 WikiText prompts, all 60 source layers, final-layer target) [27] and compared the two readouts on the 244 of 1,000 dots-10 2-fact examples the model answers correctly, taking the argmax over integer tokens at every (layer, position). Ranking among integer tokens matters: a final-layer-target J-lens on DeepSeek V3 lifts all number tokens together through the first two-thirds of the network [27], which can inflate any-token metrics but cannot affect a rank among integers. Table 16 reports the best cell for each hidden value. The J-lens surfaces the values neither earlier, at other positions, nor more reliably: the peak cell is identical in all three cases and the accuracies differ by at most 2 points.

Table 16: Logit lens vs. J-lens on DeepSeek V3, **2-fact addition**, dots-10, correct examples ( $n = 244$ ). For each hidden value, the best (layer, position) cell and the fraction of examples whose top integer token matches.

| Hidden value | Logit lens, best cell             | J-lens, best cell |
|--------------|-----------------------------------|-------------------|
| $A_1$        | 78.7% (“Filler:” label, layer 50) | 77.5% (same cell) |
| $A_2$        | 57.8% (filler dot, layer 51)      | 59.8% (same cell) |
| $A_1 + A_2$  | 88.5% (answer position, layer 59) | 88.1% (same cell) |

The sum reaches 89.8% at layer 60 under the logit lens, but that is the model’s own output distribution, where the two lenses coincide; the best pre-output cell is layer 59 for both.

## O Reasoning-trained model: Kimi K2.5

Kimi K2.5 [25] is Moonshot AI’s reasoning-trained successor to Kimi K2 (1T parameters,  $\sim 32$ B activated per token), trained with RL for chain-of-thought and offering both thinking and non-thinking modes. We use it to test whether RL training for explicit reasoning changes either the behavioral uplift or the residual-stream encoding studied in the main text.

**Setup.** Prompts follow the main-text format: 5 few-shot examples, a request for an immediate answer, and prefilled filler. Because we are studying latent rather than explicit reasoning, we additionally prefill an empty `<think></think>` block to suppress the model’s explicit reasoning. Hidden-state extraction and decoding use the pipeline of Section 5 unchanged.

**Behavioral uplift.** With  $n = 1,000$  examples per task and condition, Kimi K2.5 shows uplift on 2-fact addition ( $39\% \rightarrow 48\%$ ) and on the system of equations ( $31\% \rightarrow 46\%$ ), comparable in size to the uplift on Kimi K2.

**Model-generated filler.** The main-text setup prefills the filler. To check that the effect does not depend on the user supplying the filler, we let Kimi K2.5 generate freely and produce the filler itself before answering. The few-shot examples show filler followed immediately by an answer, and the system prompt states: “Do not explain your work and do not reason step by step in words.” Not every generation followed the format: of 1,300 attempts on the system-of-equations task, 21% skipped the dots and answered immediately, 7% produced the wrong number of dots, and 1% ignored the instruction and reasoned explicitly. On the 927 generations that followed the format, accuracy is 28.7% at baseline (no filler), 39.2% with 25 self-generated dots, and 45.8% with 25 self-generated counting tokens (Table 17). Uplift therefore remains when the dots are model-generated rather than prefilled. Few-shot examples are still required in the prompt, but they cannot explain the effect: the few-shot structure is the same in every condition, while the uplift varies with filler type (scrambled filler underperforms by  $\sim 10$  points), with length (rising, asymptoting, then degrading), and with position (filler before the question yields nothing).

**Hidden-state structure and decoding.** The logit-lens readouts on Kimi K2.5 show the same residual-stream structure as Kimi K2 and DeepSeek V3, with the addends encoded at distinct filler positions and the sum crystallizing in late layers. Running the unsupervised decoder on 2-fact addition, both  $A_1$  and  $A_2$  appear among the top-2 numeric tokens in 80.6% of correct examples ( $n = 962$ , dots-10 and dots-25 pooled), without a judge. This is comparable to Kimi K2 (74.1% pooled) and well above DeepSeek V3 (35.2%). RL training for chain-of-thought does not appear to change how this computation is encoded in the residual stream, at least when explicit reasoning is suppressed.

Table 17: Kimi K2.5 summary. *Uplift* rows compare no-filler baseline with the best filler condition ( $n = 1,000$  per task and condition, prefilled filler, `<think></think>` prefilled). *Self-generated* rows are on the system of equations with the model producing its own filler ( $n = 927$  format-following generations of 1,300). *Decoding* is direct top-2 recovery of both addends on 2-fact addition ( $n = 962$ , dots-10 and dots-25 pooled, no judge).

| Experiment                            | Task                | Baseline (%) | With filler (%) |
|---------------------------------------|---------------------|--------------|-----------------|
| Uplift (prefilled)                    | 2-fact addition     | 39           | 48              |
| Uplift (prefilled)                    | System of equations | 31           | 46              |
| Self-generated, 25 dots               | System of equations | 28.7         | 39.2            |
| Self-generated, 25 counting           | System of equations | 28.7         | 45.8            |
| Decoding (direct top-2, both addends) | 2-fact addition     | 80.6         |                 |